

İST347 İstatistiksel Analiz Uygulamaları

Kod

Ders Notları — 1.–6. Haftalar (2026–2027 Güz)

YAZAR
Dr. Öğr. Üyesi Onur TokaKURUMSAL ÜYELİK
Hacettepe Üniversitesi, İstatistik BölümüYAYINLANMA TARİHİ
28 Eylül 2026

İST347 İstatistiksel Analiz Uygulamaları ders notları **Quarto** ile hazırlanmıştır. Bu dosyadaki bilgiler ve veriler izin ve bilgi dahilinde paylaşılabilir. Quarto hakkında ayrıntılı bilgi için <https://quarto.org>, R Markdown için <https://rmarkdown.rstudio.com> adreslerini ziyaret edebilirsiniz.

Bu notlar nasıl kullanılır?

- Dosya, ders klasöründeki DERS_NOTLARI/RMARKDOWN/ içinde bulunan `ist347.Rproj` projesi açılarak çalıştırılmalıdır. Ekran görüntüleri `img/` klasöründen, veri dosyaları `./VERI_KUMELERI/` klasöründen **görelî yol** ile okunur. Böylece dosya hangi bilgisayara kopyalanırsa kopyalansın çalışır (bkz. [Çalışma dizini ve görelî yollar](#)).
- Her haftanın sonunda **Alıştırmalar** bölümü vardır. Kodları önce kendiniz yazıp çalıştırmanız önerilir.
- Ekran görüntüleri RStudio'nun eski bir sürümünden alınmıştır; menülerin yeri güncel sürümde küçük farklılıklar gösterebilir.

Haftalık plan (bu dosyada yer alan kısım)

Hafta	Konu
1	R & RStudio: arayüz, paketler, temel sözdizimi, yardım
2	Veri yapıları: vektör, matris, faktör, liste, data frame
3	Raporlama: Quarto (& R Markdown)
4	Veri okuma/yazma + veri düzenleme (dplyr)
5	Kontrol yapıları (if/else, for/while) + fonksiyon yazma
6	Görselleştirme: base R, ggplot2 ve plotly

1 Hafta 1 — R ve RStudio

Bu hafta neler öğreneceğiz?

R ve RStudio'nun ne olduğunu, RStudio ekranındaki pencereleri, paket kurma/yükleme, yardım alma, R'ı hesap makinesi gibi kullanma, değişken tanımlama ve temel veri tipleri.

1.1 R Programı

R, istatistiksel hesaplama ve grafik için geliştirilmiş **ücretsiz ve açık kaynak** bir programlama dilidir. Ross Ihaka ve Robert Gentleman tarafından 1993 yılında Yeni

Zelanda'daki Auckland Üniversitesi'nde geliştirilmeye başlanmış; bugün **R Core Team** tarafından sürdürülmektedir. R, Bell Laboratuvarları'nda geliştirilen **S** dilinin açık kaynak bir uyarlaması olarak düşünülebilir.

1.1.1 R Hakkında Genel Bilgiler

- Veri düzenleme (manipülasyon), grafiksel çözümleme, istatistiksel analiz ve hesaplamalar için bütünsel bir ortamdır.
- İşlemler **vektör ve matris** mantığıyla yapılır; yani bir işlemi tek tek elemanlara değil, bütün veriye aynı anda uygulayabilirsiniz.
- Grafikler doğrudan ekrana çizdirilebileceği gibi farklı uzantılarla (png, pdf, svg vb.) kaydedilebilir.
- Koşullar, döngüler ve kendi fonksiyonlarınızı yazmak için basit ve etkili bir programlama dili sunar.
- **CRAN** (Comprehensive R Archive Network) üzerinde 20.000'den fazla ek paket bulunur. Paketlerin çoğu belirli bir analize ya da işe hizmet etmek için yazılmıştır.

1.1.2 R Programını İndirme

<https://cran.r-project.org/> adresinden işletim sisteminize (Windows / macOS / Linux) uygun R sürümünü indirebilirsiniz. **Önce R, sonra RStudio** kurulmalıdır; RStudio kendi başına R içermez, bilgisayardaki R'ı kullanır.

1.2 RStudio

RStudio, R ile çalışmayı kolaylaştıran bir **bütünsel geliştirme ortamıdır** (IDE, *Integrated Development Environment*). Kısaca: **R motordur, RStudio ise gösterge paneli**. Kodları R çalıştırır; RStudio bize editör, grafik penceresi, dosya gezgini, yardım gibi araçları tek ekranda sunar.

1.2.1 RStudio Hakkında Genel Bilgiler

- RStudio, Joseph J. Allaire tarafından kurulan şirket tarafından geliştirilmiş, ilk sürümü Şubat 2011'de çıkmıştır.
- Şirketin adı 2022'de **Posit** olarak değişmiştir; program hâlâ “RStudio” adıyla dağıtılmaktadır.
- Aynı ekibin geliştirdiği önemli R paketleri: **tidyverse** (dplyr, ggplot2, readr...), **shiny**, **rmarkdown**, **knitr**, **devtools**. Aynı ekip **Quarto** raporlama sistemini de geliştirmiştir.

1.2.2 RStudio Programını İndirme

<https://posit.co/download/rstudio-desktop/> adresinden ücretsiz “RStudio Desktop” sürümünü indirebilirsiniz.

1.2.3 RStudio Ekran Görünümü

RStudio açıldığında ekran genellikle **dört bölmeye** ayrılır:

Bölme	Konum (varsayılan)	Ne işe yarar?
Editor (Source)	Sol üst	Kod dosyalarını (.R , .qmd , .Rmd) yazıp kaydettiğimiz yer
Console	Sol alt	Kodların gerçekten

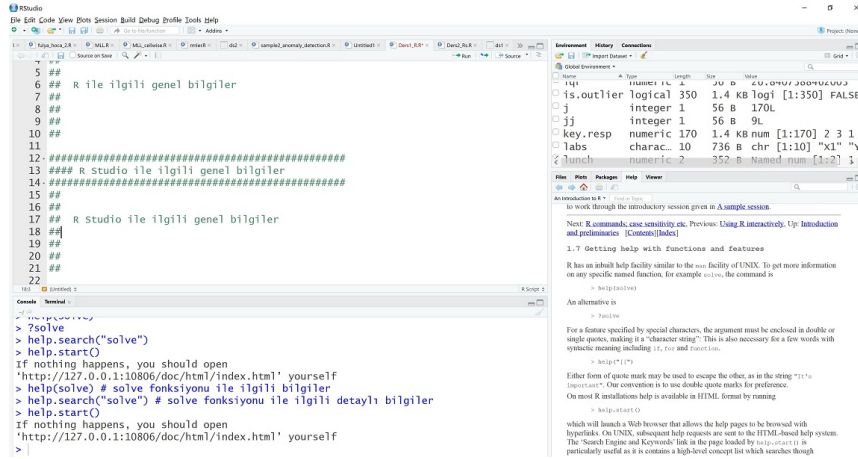
çalıştığı yer; sonuçlar burada görünür

Environment / History Sağ üst

Oluşturulan nesneler ve daha önce çalıştırılan kodlar

Files / Plots / Packages / Help / Viewer Sağ alt

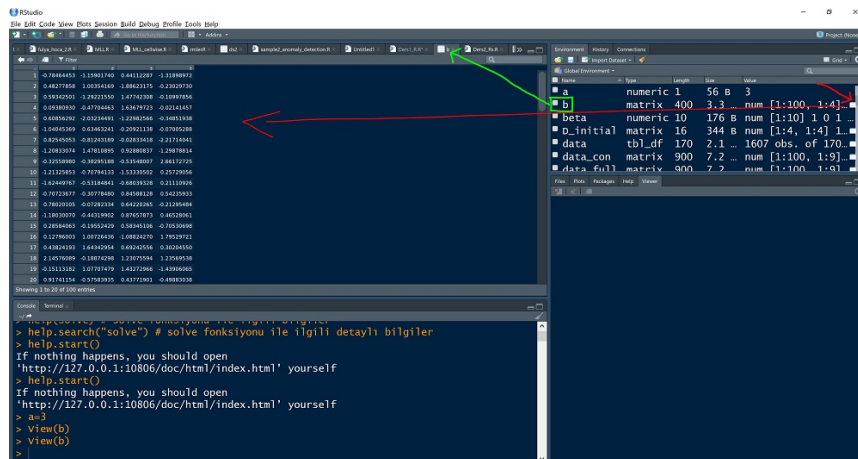
Dosyalar, grafikler, paketler, yardım sayfaları



Şekil 1: RStudio ekran görünümü

1.2.4 RStudio Program Penceresi

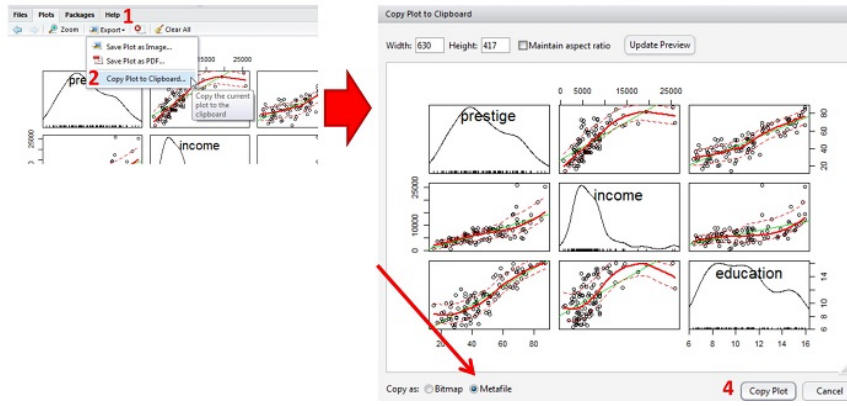
- **Console:** Kod yazıp *Enter* tuşuna bastığınız anda kod çalışır. Denemeler için uygundur ama yazılanlar kaydedilmez.
- **Editor:** Kodları bir dosyaya yazıp kaydedebilirsiniz. Yazılan kod, çalıştırılmadıkça (*Run* düğmesi ya da **Ctrl + Enter** / Mac'te **Cmd + Enter**) çalışmaz. **Kalıcı olması gereken her kod editörde yazılmalıdır.**
- **Environment (Workspace):** Çalıştırılan kodlarla üretilmiş tüm nesneleri (sayı, vektör, karakter, matris, veri tablosu, fonksiyon vb.) gösterir. Bir veri tablosunun yanındaki tablo simgesine tıklanarak veri incelenebilir (*View()* fonksiyonu ile aynı işi görür).



Şekil 2: Environment penceresi ve View() ile veri inceleme

- **History:** Daha önce çalıştırılmış kodları gösterir. Buradan kodlar editöre ya da konsola gönderilebilir.
- **Plots:** Çizdirilen grafikler burada görünür. Birden fazla grafik varsa sağ/sol oklarla aralarında geçiş yapılabilir. **Export** menüsü ile grafikler resim ya da pdf

olarak kaydedilebilir veya panoya kopyalanabilir.



Şekil 3: Plots penceresinden grafiği dışa aktarma

- **Packages:** Bilgisayarda kurulu paketleri listeler. Yanındaki kutucuk işaretliyse paket o oturumda **yüklenmiş** (kullanıma hazır) demektir.

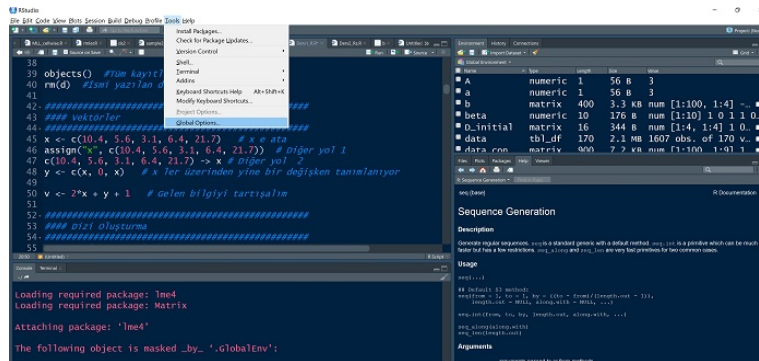


Şekil 4: Packages penceresi

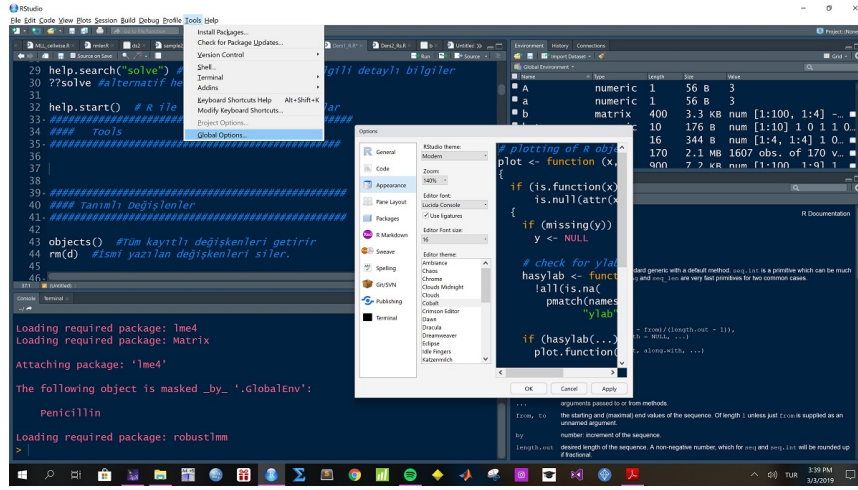
- **Help:** Paketler ve fonksiyonlarla ilgili açıklamalar, argümanlar ve örnek kodlar burada görünür.
- **Viewer:** Etkileşimli grafikler (plotly) ve Quarto/R Markdown çıktıları burada önizlenir.
- **Connections:** Veri tabanı bağlantılarının yönetildiği alandır.
- **Git:** Kodların Git/GitHub üzerinden sürüm kontrolü ve ekip çalışması için kullanılır (projede Git etkinse görünür).
- **Tutorial:** learnr paketi ile hazırlanmış etkileşimli derslerin açıldığı sekmedir.

1.2.5 Görünümü Değiştirme

Tools → **Global Options** menüsünden yazı tipi, tema (açık/koyu), bölmelerin yerleşimi gibi ayarlar değiştirilebilir. *Appearance* sekmesinden editör teması, *Pane Layout* sekmesinden bölmelerin yeri ayarlanır.



Şekil 5: Tools menüsü



Şekil 6: Global Options → Appearance

1.3 Projeler, Çalışma Dizini ve Görelî Yollar

R bir dosyayı okurken ya da kaydederken **çalışma dizinine** (*working directory*) bakar.

```
getwd()           # Şu anki çalışma dizinini gösterir
setwd("C:/Kullanici/Belgeler/ist347") # Çalışma dizinini değiştirir
list.files()       # Çalışma dizinindeki dosyaları listeler
```

Mutlak yol yerine görelî yol kullanın

"D:/Drive_Dosyalar/2024-25_DERSLER/.../r1.JPG" gibi **mutlak yollar** yalnızca o bilgisayarda çalışır. Dosya başka bir bilgisayara, başka bir klasöre ya da Mac'e taşındığında kod hata verir.

Çözüm:

1. **File** → **New Project** ile bir **RStudio projesi** (.Rproj) oluşturun ve çalışmaya her zaman bu dosyayı açarak başlayın. Proje açıldığında çalışma dizini otomatik olarak proje klasörü olur.
2. Dosyaları proje klasörüne göre **görelî yol** ile çağırın: "img/r1.jpg" , "../VERI_KUMELERI/worms.txt" gibi. (.. "bir üst klasör" demektir.)
3. Windows'ta yol yazarken ters bölü \ yerine düz bölü / kullanın.
4. Quarto/R Markdown belgesi derlenirken çalışma dizini **belgenin bulunduğu klasördür**.

Bu dersin klasör yapısı:

```
ist347/
├── DERS_NOTLARI/
│   ├── RMARKDOWN/
│   │   ├── ist347.Rproj
│   │   ├── ist347_ders_notlari.qmd  <- bu dosya
│   │   └── img/                    <- ekran görüntüleri
│   └── (r1.jpg ... r10.jpg)
└── VERI_KUMELERI/
    ├── worms.txt
    └── dataset_ex.csv
```

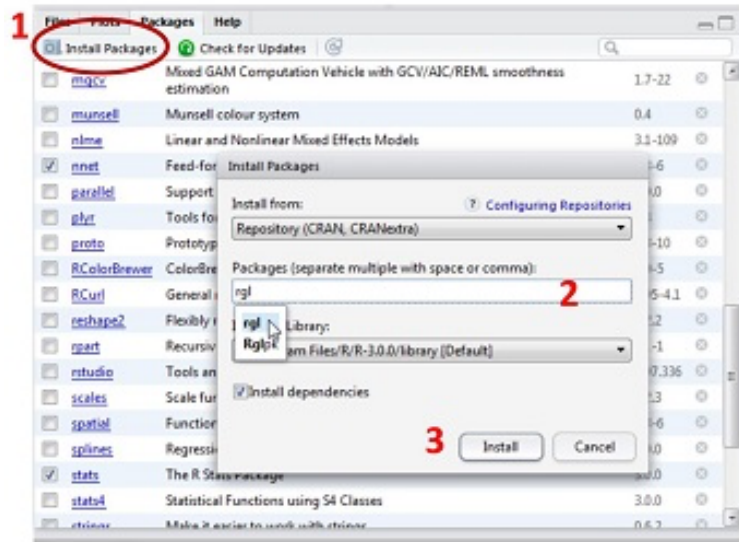
1.4 Paketler

R'ın temel kurulumunda gelen fonksiyonlara **base R** denir. Ek özellikler **paketler** ile gelir. Paketlerle çalışırken iki adımı ayırt etmek gerekir:

Adım	Ne zaman?	Kod
Kurma (install)	Bilgisayara bir kez	<code>install.packages("dplyr")</code>
Yükleme (load)	Her yeni oturumda	<code>library(dplyr)</code>

Benzetme: `install.packages()` kitabı satın alıp rafa koymak, `library()` ise kitabı raftan alıp masaya açmaktır.

Paketler, **Packages** → **Install** düğmesiyle menüden de kurulabilir:



Şekil 7: Packages → Install ile paket kurma

```
install.packages(c("dplyr", "ggplot2", "readr", "readxl", "p...t")
library(dplyr) # her oturumda
dplyr::filter # Paketi yüklemeyen tek bir fonksiyonu l
```

Not

Farklı paketlerde aynı isimli fonksiyonlar olabilir. Bir paket yüklenirken konsolda “*The following objects are masked from ...*” uyarısı görürseniz, son yüklenen paketin fonksiyonu öncekini “örtmüş” demektir. Bu durumda `stats::filter()` ya da `dplyr::filter()` şeklinde hangi paketin fonksiyonunu istediğinizi açıkça yazabilirsiniz.

1.5 Yardım Almak

- `help(solve)` veya `?solve`: `solve` fonksiyonunun yardım sayfasını açar.
- `help.search("solve")` veya `??solve`: İçinde “solve” geçen tüm yardım sayfalarında arama yapar (fonksiyonun tam adını bilmiyorsanız).
- `example(mean)`: Yardım sayfasındaki örnekleri çalıştırır.

- `help.start()` : R'in genel yardım ve kılavuz sayfalarını açar.
- `vignette("dplyr")` : Bir paketin uzun anlatımlı kullanım kılavuzunu açar.

Yardım sayfalarında en çok işe yarayan kısımlar: **Usage** (nasıl çağrılır), **Arguments** (argümanlar), **Value** (ne döndürür) ve **Examples** (örnekler).

1.6 Tanımlı Nesneler ile Çalışma

- `ls()` ya da `objects()` : Tüm kayıtlı nesneleri listeler.
- `rm(x)` : İsmi yazılan nesneyi siler. `rm(list = ls())` hepsini siler.
- `getwd()` / `setwd()` / `list.files()` : Bkz. Çalışma dizini.

1.7 R'ı Hesap Makinesi Olarak Kullanma

- Konsola ya da editöre yazılan işlemler çalıştırılarak hesaplamalar yapılabilir. # işaretinden sonra yazılanlar **yorumdur**, çalıştırılmaz.

```
3 + 5      # Toplama
```

```
[1] 8
```

```
9 - 2      # Çıkarma
```

```
[1] 7
```

```
2 * 3      # Çarpma
```

```
[1] 6
```

```
9 / 4      # Bölme
```

```
[1] 2.25
```

```
2^10       # Üs alma
```

```
[1] 1024
```

```
17 %/% 5   # Tam bölme
```

```
[1] 3
```

```
17 %% 5    # Mod (kalan)
```

```
[1] 2
```

```
log(exp(5)) # Doğal logaritma
```

[1] 5

```
log10(100)
```

[1] 2

```
exp(2)
```

[1] 7.389056

```
sqrt(100)
```

[1] 10

```
round(pi, 3)
```

[1] 3.142

- Mantıksal işlemlerin cevabı TRUE ya da FALSE olarak alınır.

```
# Karşılaştırma: <, <=, >, >=, == (eşit mi?), != (eşit değil mi?)  
# Birleştirme: c1 & c2 (ve), c1 | c2 (veya), !c1 (değil)
```

```
3 > 13
```

[1] FALSE

```
!2 > 13
```

[1] TRUE

```
!(5 < 2) & (8 != 4)
```

[1] TRUE

```
(1 > 5) | (4 == 5)
```

[1] FALSE

Önemli

= atama, == karşılaştırma içindir. `x == 3` “x 3’e eşit mi?” diye sorar; `x = 3` ise x’e 3 değerini verir.

1.8 Değişkenlere Değer Atama

- R’da atama için en yaygın kullanılan işaret `<-` işaretidir (RStudio’da kısayolu **Alt + -**). `=` de çalışır.

```
x <- c(10.4, 5.6, 3.1, 6.4, 21.7)      # x'e ata
assign("x1", c(10.4, 5.6, 3.1, 6.4, 21.7)) # Diğer yol 1
c(10.4, 5.6, 3.1, 6.4, 21.7) -> x2      # Diğer yol 2
x3 = c(10.4, 5.6, 3.1, 6.4, 21.7)      # Diğer yol 3
y <- c(x, x)                          # x'ler üzerinden yeni bir değişken tanımla
v <- 2*x + y + 1                       # Gelen bilgiyi tartışalım: x 5, y 10 elemanlı
v
```

```
[1] 32.2 17.8 10.3 20.2 66.1 32.2 17.8 10.3 20.2 66.1
```

```
x                                # Veriyi görmek için adı yazılabilir
```

```
[1] 10.4 5.6 3.1 6.4 21.7
```

```
print(x)                        # Ya da print() kullanılabilir
```

```
[1] 10.4 5.6 3.1 6.4 21.7
```

Geri dönüşüm (recycling) kuralı

Yukarıda x 5, y 10 elemanlıdır. R, kısa olan vektörü uzun olanın boyuna ulaşana kadar **tekrar eder**. Uzunluklar birbirinin katı değilse işlem yine yapılır ama **uyarı** verilir:

```
y1 <- c(x, 0, x)                # 11 elemanlı
v1 <- 2*x + y1 + 1              # 11, 5'in katı değil -> uyarı
```

Warning in 2 * x + y1: longer object length is not a multiple of shorter object length

```
v1
```

```
[1] 32.2 17.8 10.3 20.2 66.1 21.8 22.6 12.8 16.9 50.8 43.5
```

Değişken isimlendirme kuralları: Harf ya da nokta ile başlar; rakam ile başlayamaz; boşluk içeremez; büyük/küçük harfe duyarlıdır (Veri ≠ veri). Okunabilirlik için `ogrenci_notu` gibi alt çizgili isimler önerilir.

1.9 Temel Veri Tipleri

Tip	Örnek	class() sonucu
Mantıksal	TRUE, FALSE	"logical"
Sayısal (ondalıklı)	3, 4.8	"numeric"
Tam sayı	5L	"integer"
Karakter	"Karakter"	"character"
Eksik değer	NA	(tipine göre)

```
a <- TRUE
b <- 3
c <- 4.8
d <- 5L
e <- "Karakter"

class(a); class(b); class(c); class(d); class(e)
```

```
[1] "logical"
```

```
[1] "numeric"
```

```
[1] "numeric"
```

```
[1] "integer"
```

```
[1] "character"
```

```
str(a); str(b); str(c); str(d); str(e)
```

```
logi TRUE
```

```
num 3
```

```
num 4.8
```

```
int 5
```

```
chr "Karakter"
```

- Tip dönüşümleri `as.` ile başlayan fonksiyonlarla, tip sorgulama `is.` ile başlayan fonksiyonlarla yapılır:

```
as.numeric("3.14")
```

```
[1] 3.14
```

```
as.character(42)
```

```
[1] "42"
```

```
as.integer(4.9)      # Yuvarlamaz, ondalığı atar!
```

```
[1] 4
```

```
is.numeric(b)
```

```
[1] TRUE
```

```
as.numeric("elma")   # Sayıya çevrilemez -> NA ve uyarı
```

Warning: NAs introduced by coercion

[1] NA

- Eksik değerler NA ile gösterilir. Bir değer eksik olup olmadığı `is.na()` ile sorgulanır.

```
f <- NA  
print(f)
```

[1] NA

```
is.na(f)
```

[1] TRUE

```
mean(c(4, NA, 6)) # NA varsa sonuç NA
```

[1] NA

```
mean(c(4, NA, 6), na.rm = TRUE) # NA'ları atarak hesapla
```

[1] 5

```
# Değişken ve kelimelerin birleştirilmesi  
etiket <- paste(c("X", "Y"), 1:2, sep = "")  
print(etiket)
```

[1] "X1" "Y2"

```
paste0("Hafta", 1:3)
```

[1] "Hafta1" "Hafta2" "Hafta3"

1.10 Alıştırmalar (Hafta 1)

1. Bilgisayarınıza R ve RStudio'yu kurun, `ist347` adında bir RStudio projesi oluşturun.
2. Yarıçapı 7 cm olan bir dairenin alanını R ile hesaplayıp `alan` adlı bir değişkene atayın.
3. `?round` yardım sayfasını açın. `round()` ile `signif()` arasındaki fark nedir?
4. `as.integer("12a")` ne döndürür? Neden?

2 Hafta 2 — Veri Yapıları

Bu hafta neler öğreneceğiz?

R'daki temel veri yapıları: **vektör**, **matris/dizi (array)**, **faktör**, **liste** ve **veri çerçevesi (data frame)**. Bu yapıların nasıl oluşturulduğunu ve içlerinden eleman seçmeyi

(indeksleme) öğreneceğiz.

Yapı	Boyut	Aynı tip mi olmalı?	Örnek kullanım
Vektör	1	Evet	Bir değişkenin gözlemleri
Matris	2	Evet	Korelasyon matrisi, tasarım matrisi
Dizi (array)	2+	Evet	Çok boyutlu tablolar
Faktör	1	Kategoriler	Cinsiyet, eğitim düzeyi
Liste	1	Hayır	Bir analizin tüm çıktıları
Data frame	2	Sütun içinde evet	Veri seti (satır = gözlem, sütun = değişken)

2.1 Vektörler

Vektör, **aynı tipteki** değerlerin sıralı topluluğudur ve `c()` (*combine*) fonksiyonu ile oluşturulur. R'da tek bir sayı bile aslında 1 elemanlı bir vektördür.

2.1.1 Dizi Oluşturma

- `:` ile dizi oluşturma

```
1:30 # 1'den 30'a kadar
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
20 21 22 23 24 25
[26] 26 27 28 29 30
```

```
2*1:15 # 2'den 30'a kadar çiftler çiftler
```

```
[1] 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30
```

```
30:1 # 30'dan 1'e kadar
```

```
[1] 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12
11 10 9 8 7 6
[26] 5 4 3 2 1
```

- `seq()` fonksiyonu ile dizi oluşturma

```
seq(2, 10)
```

```
[1] 2 3 4 5 6 7 8 9 10
```

```
seq(from = 1, to = 30)
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
20 21 22 23 24 25
[26] 26 27 28 29 30
```

```
seq(-5, 5, by = .2)
```

```
[1] -5.0 -4.8 -4.6 -4.4 -4.2 -4.0 -3.8 -3.6 -3.4 -3.2 -3.0
-2.8 -2.6 -2.4 -2.2
[16] -2.0 -1.8 -1.6 -1.4 -1.2 -1.0 -0.8 -0.6 -0.4 -0.2 0.0
0.2 0.4 0.6 0.8
[31] 1.0 1.2 1.4 1.6 1.8 2.0 2.2 2.4 2.6 2.8 3.0
3.2 3.4 3.6 3.8
[46] 4.0 4.2 4.4 4.6 4.8 5.0
```

```
seq(length = 51, from = -5, by = .2)
```

```
[1] -5.0 -4.8 -4.6 -4.4 -4.2 -4.0 -3.8 -3.6 -3.4 -3.2 -3.0
-2.8 -2.6 -2.4 -2.2
[16] -2.0 -1.8 -1.6 -1.4 -1.2 -1.0 -0.8 -0.6 -0.4 -0.2 0.0
0.2 0.4 0.6 0.8
[31] 1.0 1.2 1.4 1.6 1.8 2.0 2.2 2.4 2.6 2.8 3.0
3.2 3.4 3.6 3.8
[46] 4.0 4.2 4.4 4.6 4.8 5.0
```

- `rep()` fonksiyonu ile dizi oluşturma

```
s5 <- rep(x, times = 5) # Vektörün tamamını 5 kez tekrarla
s6 <- rep(x, each = 5)  # Her elemanı 5'er kez tekrarla
s5
```

```
[1] 10.4 5.6 3.1 6.4 21.7 10.4 5.6 3.1 6.4 21.7 10.4
5.6 3.1 6.4 21.7
[16] 10.4 5.6 3.1 6.4 21.7 10.4 5.6 3.1 6.4 21.7
```

```
s6
```

```
[1] 10.4 10.4 10.4 10.4 10.4 5.6 5.6 5.6 5.6 5.6 3.1
3.1 3.1 3.1 3.1
[16] 6.4 6.4 6.4 6.4 6.4 21.7 21.7 21.7 21.7 21.7
```

- Minik bir yorumlama: İşlem önceliğine dikkat!

```
n <- 10
1:n-1 # Ne verir? (önce 1:n oluşturulur, sonra her elemanı)
```

```
[1] 0 1 2 3 4 5 6 7 8 9
```

```
1:(n-1) # Ne verir?
```

```
[1] 1 2 3 4 5 6 7 8 9
```

2.1.2 Vektör Üzerinde İşlemler

İşlemler vektörün **tüm elemanlarına aynı anda** uygulanır (vektörel işlem). Döngü yazmaya gerek yoktur.

```
notlar <- c(65, 80, 45, 90, 72)
notlar + 5 # Herkese 5 puan
```

```
[1] 70 85 50 95 77
```

```
notlar > 60 # Kim geçti?
```

```
[1] TRUE TRUE FALSE TRUE TRUE
```

```
length(notlar); sum(notlar); mean(notlar); max(notlar)
```

```
[1] 5
```

```
[1] 352
```

```
[1] 70.4
```

```
[1] 90
```

```
sort(notlar)
```

```
[1] 45 65 72 80 90
```

```
sort(notlar, decreasing = TRUE)
```

```
[1] 90 80 72 65 45
```

2.1.3 İndeksleme (Eleman Seçme)

Köşeli parantez [] ile eleman seçilir. **R’da indeksleme 1’den başlar.**

Yazım	Anlamı
x[2]	2. eleman
x[c(1, 3)]	1. ve 3. elemanlar
x[-1]	1. eleman hariç hepsi
x[x > 5]	Koşulu sağlayanlar
x["isim"]	İsimli vektörlerde isimle seçim

```
y <- x[!is.na(x)] # NA olmayanlar
```

```
print(y)
```

```
[1] 10.4  5.6  3.1  6.4 21.7
```

```
(x + 1)[(!is.na(x)) & x > 1] -> z
print(z)
```

```
[1] 11.4  6.6  4.1  7.4 22.7
```

```
x[1:10] # x 5 elemanlı; olmayan elemanlar NA ile doldurulur
```

```
[1] 10.4  5.6  3.1  6.4 21.7  NA  NA  NA  NA  NA
```

```
c("x", "y")[rep(c(1, 2, 2, 1), times = 4)]
```

```
[1] "x" "y" "y" "x" "x" "y" "y" "x" "x" "y" "y" "x" "x" "y"
"y" "x"
```

```
y <- x[-(1:5)] # İlk 5 eleman hariç -> boş ve NA ile doldurulur
print(y)
```

```
numeric(0)
```

```
fruit <- c(5, 10, 1, 20)
print(fruit)
```

```
[1]  5 10  1 20
```

```
names(fruit) <- c("orange", "banana", "apple", "peach")
print(fruit)
```

```
orange banana  apple  peach
      5      10       1     20
```

```
lunch <- fruit[c("apple", "orange")]
print(lunch)
```

```
apple orange
      1      5
```

```
w <- c(-3, 4, -1, 7)
w[w < 0] <- -w[w < 0] # Negatifleri pozitiflere çevirir
print(w)
```

```
[1] 3 4 1 7
```

2.2 Matrisler ve Diziler (Array)

- `dim()` fonksiyonu ile bir nesnenin boyutları belirlenebilir ya da sorgulanabilir.
- Matris, **iki boyutlu**; array ise **iki ya da daha fazla boyutlu** yapıdır. `matrix()` fonksiyonu `nrow / ncol` argümanlarıyla, `array()` fonksiyonu `dim` argümanı ile kullanılır.
- Matrislerde eleman seçimi `[satır, sütun]` şeklindedir. Birini boş bırakmak “hepsi” anlamına gelir: `A[2, ]` 2. satırın tamamı, `A[, 3]` 3. sütunun tamamı.

```
# dim() fonksiyonu
g <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
dim(g) <- c(1, 5, 2)
dim(g) <- c(5, 2, 1)

# Deneyelim
g[, , 1]
```

```
      [,1] [,2]
[1,]     1     6
[2,]     2     7
[3,]     3     8
[4,]     4     9
[5,]     5    10
```

```
g[5, 1, 1]
```

```
[1] 5
```

```
# array() fonksiyonu
x <- array(1:20, dim = c(4, 5))          # 4x5 boyutlu matris
i <- array(c(1:3, 3:1), dim = c(3, 2))    # 3x2 boyutlu indeksler
x[i]          # i matrisindeki (satır, sütun) gözlerini x'ten çek
```

```
[1] 9 6 3
```

```
x[i] <- 0 # 0 gözlerle 0 değerini ver
x          # x'i tekrar yazdır
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,]     1     5     0    13    17
[2,]     2     0    10    14    18
[3,]     0     7    11    15    19
[4,]     4     8    12    16    20
```

- `seq()`, `rep()` gibi fonksiyonlar ile oluşturulan diziler matris ya da array haline getirilebilir.

```
h <- 1:48
Z1 <- array(h, dim = c(3, 4, 2))          # 3x4'lük iki katman
Z2 <- array(0, c(3, 4, 2))                # Sıfır dizi oluşturma
Z3 <- matrix(h, nrow = 4)                  # 4 satırlı matris (sütun sütun)
Z4 <- matrix(h, nrow = 8, byrow = TRUE)    # Satır satır doldurma
Z1; Z2; Z3; Z4
```

```
, , 1
```

```
      [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

```
, , 2
```

```
      [,1] [,2] [,3] [,4]
[1,]   13   16   19   22
[2,]   14   17   20   23
[3,]   15   18   21   24
```

```
, , 1
```

```
      [,1] [,2] [,3] [,4]
[1,]     0     0     0     0
[2,]     0     0     0     0
[3,]     0     0     0     0
```

```
, , 2
```

```
      [,1] [,2] [,3] [,4]
[1,]     0     0     0     0
[2,]     0     0     0     0
[3,]     0     0     0     0
```

```
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11]
[1,]    1    5    9   13   17   21   25   29   33   37   41
45
[2,]    2    6   10   14   18   22   26   30   34   38   42
46
[3,]    3    7   11   15   19   23   27   31   35   39   43
47
[4,]    4    8   12   16   20   24   28   32   36   40   44
48
```

```
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]     1     2     3     4     5     6
[2,]     7     8     9    10    11    12
[3,]    13    14    15    16    17    18
[4,]    19    20    21    22    23    24
[5,]    25    26    27    28    29    30
[6,]    31    32    33    34    35    36
[7,]    37    38    39    40    41    42
[8,]    43    44    45    46    47    48
```

Not

`matrix()` varsayılan olarak değerleri **sütun sütun** yerleştirir. Satır satır doldurmak için `byrow = TRUE` kullanılır.

- Matrisler istatistiksel işlemlerde çok önemli yer tutar (regresyon, kovaryans, özdeğer vb.). Bazı örnekler:

```
# Devrik (transpose)
A <- array(1:9, dim = c(3, 3))
B <- t(A)
print(A); print(B)
```

```
      [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
# Satır ve sütun sayısı
nrow(A); ncol(A); dim(A)
```

```
[1] 3
```

```
[1] 3
```

```
[1] 3 3
```

```
# Çarpım
A * B      # Eleman eleman çarpım
```

```
      [,1] [,2] [,3]
[1,]    1    8   21
[2,]    8   25   48
[3,]   21   48   81
```

```
A %*% B      # Matris çarpımı
```

```
      [,1] [,2] [,3]
[1,]   66   78   90
[2,]   78   93  108
[3,]   90  108  126
```

```
diag(A)      # Köşegen elemanlar
```

```
[1] 1 5 9
```

```
# Denklem sistemi çözme örneği
# x + y + z = 3
# x + 2y + 3z = 0
# x + 3y + 4z = -2
```

```
A <- array(c(1, 1, 1, 1, 2, 3, 1, 3, 4), dim = c(3, 3)); b <- c(5, -1, -1)
solve(A, b)
```

```
[1] 5 -1 -1
```

```
solve(A) %*% b      # A'nın tersi ile b'nin çarpımı -> aynı sonuç
```

```
      [,1]
[1,] 5
[2,] -1
[3,] -1
```

```
# Özdeğer bulma örneği
B <- A %*% t(A)
ozdeger1 <- eigen(B)
ozdegerler <- ozdeger1$values
print(B); print(ozdeger1); print(ozdegerler)
```

```
      [,1] [,2] [,3]
[1,] 3 6 8
[2,] 6 14 19
[3,] 8 19 26
```

```
eigen() decomposition
$values
[1] 42.45885379 0.49341294 0.04773327
```

```
$vectors
      [,1]      [,2]      [,3]
[1,] -0.2456499 0.93905997 0.2404631
[2,] -0.5735327 0.05918694 -0.8170417
[3,] -0.7814834 -0.33861965 0.5240424
```

```
[1] 42.45885379 0.49341294 0.04773327
```

- **Örnek soru:** Tekil değer ayrışımı (*Singular Value Decomposition*) fonksiyonunu kullanarak B matrisini ayrıştıran matrisleri bulunuz. (İpucu: `??"singular value"`)
- Matrislerde satır ve sütun birleştirme `rbind()` (*row bind*) ve `cbind()` (*column bind*) ile yapılır:

```
x1 <- array(0, 3)
x2 <- array(1, 3)
x3 <- cbind(1, x1, x2)
x4 <- rbind(x1, x2)
x5 <- cbind(x1, x2)
print(x3); print(x4); print(x5)
```

```
      x1 x2
[1,] 1 0 1
[2,] 1 0 1
```

```
[3,] 1 0 1

      [,1] [,2] [,3]
x1      0    0    0
x2      1    1    1

      x1 x2
[1,]  0  1
[2,]  0  1
[3,]  0  1
```

2.3 Faktörler: Nominal ve Ordinal Yapılar

İstatistiksel analizde yapılacak analizler, değişkenin **ölçme düzeyine** göre değişir. Kategorik değişkenler R’da **faktör** olarak saklanır:

- **Nominal** (sınıflama): Kategoriler arasında sıralama yoktur (cinsiyet, il, kan grubu).
- **Ordinal** (sıralama): Kategoriler arasında doğal bir sıra vardır (eğitim düzeyi, unvan, memnuniyet: az < orta < çok).

Faktörün alabileceği kategorilere **düzy** (*level*) denir.

```
# Faktör elde etme
cinsiyet <- c("Kadın", "Kadın", "Erkek", "Kadın", "Erkek", "Erkek", "Kadın", "Erkek")
cinsiyet_faktor <- factor(cinsiyet)
print(cinsiyet); print(cinsiyet_faktor)
```

```
[1] "Kadın" "Kadın" "Erkek" "Kadın" "Erkek" "Erkek" "Kadın" "Erkek"
```

```
[1] Kadın Kadın Erkek Kadın Erkek Erkek Kadın Erkek
Levels: Erkek Kadın
```

```
# Düzeyleri görme (nominal faktör; varsayılan sıralama alfab. ile)
levels(cinsiyet_faktor)
```

```
[1] "Erkek" "Kadın"
```

```
table(cinsiyet_faktor)
```

```
cinsiyet_faktor
Erkek Kadın
      4      4
```

```
# Ordinal faktör: sırayı levels ile biz veriyoruz
gorev_unvan <- c("Genel Müdür", "Genel Müdür", "Uzman", "Uzman", "Uzman", "Uzman", "Uzman Yardımcısı", "Uzman Yardımcısı", "Uzman Yardımcısı", "Uzman Yardımcısı", "Danışman", "Danışman", "Danışman", "Danışman", "Danışman", "Uzman", "Uzman", "Uzman", "Uzman Yardımcısı")
gorev_unvan_faktor <- factor(gorev_unvan, ordered = TRUE,
```

```
levels = c("Uzman Yardımcısı", "Uzman", "Danışman", "Genel Müdür")
print(gorev_unvan); print(gorev_unvan_faktor)
```

```
[1] "Genel Müdür"      "Genel Müdür"      "Uzman"
"Uzman"
[5] "Uzman"            "Uzman"            "Uzman Yardımcısı"
"Uzman Yardımcısı"
[9] "Uzman Yardımcısı" "Danışman"         "Danışman"
"Danışman"
[13] "Danışman"         "Uzman"            "Uzman"
"Uzman"
[17] "Uzman Yardımcısı"

[1] Genel Müdür      Genel Müdür      Uzman            Uzman
[5] Uzman            Uzman            Uzman Yardımcısı Uzman
Yardımcısı
[9] Uzman Yardımcısı Danışman         Danışman
Danışman
[13] Danışman         Uzman            Uzman            Uzman
[17] Uzman Yardımcısı
Levels: Uzman Yardımcısı < Uzman < Danışman < Genel Müdür
```

```
gorev_unvan_faktor[1] > gorev_unvan_faktor[3] # Ordinal fa ö
```

```
[1] TRUE
```

- **Örnek veri kümesi:** Avustralya eyaletlerinden 30 vergi mükellefinin eyalet ve gelir bilgisi.

```
state <- c("tas", "sa", "qld", "nsw", "nsw", "nt", "wa", "wa", "qld", "vic", "nsw", "vic", "qld", "qld", "sa", "tas", "sa", "nt", "wa", "vic", "qld", "nsw", "nsw", "wa", "sa", "act", "nsw", "vic", "vic", "act")
statef <- factor(state)
levels(statef)
```

```
[1] "act" "nsw" "nt"  "qld" "sa"  "tas" "vic" "wa"
```

```
incomes <- c(60, 49, 40, 61, 64, 60, 59, 54, 62, 69, 70, 42, 61, 61, 61, 58, 51, 48, 65, 49, 49, 41, 48, 52, 46, 59, 46, 58, 43)

# Sürekli değişkeni cut() ile sınıflara ayırıp faktöre çevirme
incomef1 <- cut(incomes, breaks = 35 + 10*(0:4))
incomef2 <- cut(incomes, breaks = 35 + 10*(0:4), ordered_result = TRUE)
print(incomes); print(incomef1); print(incomef2)
```

```
[1] 60 49 40 61 64 60 59 54 62 69 70 42 56 61 61 61 58 51 48
65 49 49 41 48 52
[26] 46 59 46 58 43
```

```
[1] (55,65] (45,55] (35,45] (55,65] (55,65] (55,65] (55,65]
```

```
(45,55] (55,65]
[10] (65,75] (65,75] (35,45] (55,65] (55,65] (55,65] (55,65]
(55,65] (45,55]
[19] (45,55] (55,65] (45,55] (45,55] (35,45] (45,55] (45,55]
(45,55] (55,65]
[28] (45,55] (55,65] (35,45]
Levels: (35,45] (45,55] (55,65] (65,75]
```

```
[1] (55,65] (45,55] (35,45] (55,65] (55,65] (55,65] (55,65]
(45,55] (55,65]
[10] (65,75] (65,75] (35,45] (55,65] (55,65] (55,65] (55,65]
(55,65] (45,55]
[19] (45,55] (55,65] (45,55] (45,55] (35,45] (45,55] (45,55]
(45,55] (55,65]
[28] (45,55] (55,65] (35,45]
Levels: (35,45] < (45,55] < (55,65] < (65,75]
```

```
table(incomef2, statef) # Çapraz tablo
```

```
      statef
incomef2 act nsw nt qld sa tas vic wa
(35,45]   1  1  0  1  0  0  1  0
(45,55]   1  1  1  1  2  0  1  3
(55,65]   0  3  1  3  2  2  2  1
(65,75]   0  1  0  0  0  0  1  0
```

```
# tapply(): Bir değişkeni faktörün düzeylerine göre özetler
tapply(incomes, statef, mean) # Eyaletlere göre ortalama gelir
```

```
      act      nsw      nt      qld      sa      tas      vic
wa
44.50000 57.33333 55.50000 53.60000 55.00000 60.50000 56.00000
52.25000
```

2.4 Listeler

Liste; sayı, vektör, matris, veri tablosu, hatta başka listeler gibi **farklı tipteki** nesneleri bir arada tutabilen yapıdır. R'daki pek çok analiz fonksiyonu (örneğin `t.test()`, `lm()`) sonuçlarını liste olarak döndürür.

```
Lst <- list(İsim = "Ahmet", Eşi = "Ayşe", çocuk.say = 3,
           çocuk.yaş = c(4, 7, 9))
Lst
```

```
$İsim
[1] "Ahmet"
```

```
$Eşi
[1] "Ayşe"
```

```
$çocuk.say
```

```
[1] 3
```

```
$çocuk.yaş
```

```
[1] 4 7 9
```

```
Lst[[1]]
```

```
[1] "Ahmet"
```

```
Lst[[2]]
```

```
[1] "Ayşe"
```

```
Lst[[3]]
```

```
[1] 3
```

```
Lst[[4]]
```

```
[1] 4 7 9
```

```
Lst[[4]][1]
```

```
[1] 4
```

```
length(Lst)
```

```
[1] 4
```

```
Lst$İsim
```

```
[1] "Ahmet"
```

```
Lst$Eşi
```

```
[1] "Ayşe"
```

```
Lst[5] <- list("evet")  
str(Lst)
```

```
List of 5
```

```
$ İsim      : chr "Ahmet"
```

```
$ Eşi       : chr "Ayşe"
```

```
$ çocuk.say: num 3
```

```
$ çocuk.yaş: num [1:3] 4 7 9
```

```
$          : chr "evet"
```

[] ile [[]] farkı

Lst[1] sonucu yine bir **listedir** (içinde tek eleman olan bir liste, “kutunun içindeki kutu”). Lst[[1]] ise o elemanın **kendisini** verir (“kutunun içindeki eşya”). Lst\$İsim ifadesi Lst[["İsim"]] ile aynıdır.

2.5 Veri Çerçevesi (Data Frame)

Data frame, istatistikte en çok kullanılan veri yapısıdır: her **satır bir gözlem**, her **sütun bir değişkendir**. Farklı sütunlar farklı tipte olabilir (biri sayısal, biri faktör...) ama tüm sütunlar aynı uzunluktadır. Teknik olarak data frame, eşit uzunluklu vektörlerden oluşan bir listedir.

```
ogrenci <- data.frame(
  ad      = c("Ali", "Ayşe", "Can", "Deniz", "Ece"),
  bolum   = factor(c("İst", "Akt", "İst", "İst", "Akt")),
  vize    = c(55, 80, 40, 90, 72),
  final   = c(60, 85, 50, 88, 65)
)
ogrenci
```

ad	bolum	vize	final
<chr>	<fct>	<dbl>	<dbl>
Ali	İst	55	60
Ayşe	Akt	80	85
Can	İst	40	50
Deniz	İst	90	88
Ece	Akt	72	65

5 rows

```
str(ogrenci)          # Yapı: sütun adları ve tipleri
```

```
'data.frame':  5 obs. of  4 variables:
 $ ad      : chr  "Ali" "Ayşe" "Can" "Deniz" ...
 $ bolum   : Factor w/ 2 levels "Akt","İst": 2 1 2 2 1
 $ vize    : num  55 80 40 90 72
 $ final   : num  60 85 50 88 65
```

```
dim(ogrenci)          # Satır ve sütun sayısı
```

```
[1] 5 4
```

```
names(ogrenci)        # Sütun adları
```

```
[1] "ad"      "bolum"   "vize"     "final"
```

```
head(ogrenci, 3)      # İlk 3 satır
```

ad	bolum	vize	final
<chr>	<fct>	<dbl>	<dbl>

1	Ali	İst	55	60
2	Ayşe	Akt	80	85
3	Can	İst	40	50

3 rows

```
summary(ogrenci)      # Her sütunun özeti
```

```

      ad      bolum      vize      final
Length:5      Akt:2  Min.   :40.0  Min.   :50.0
Class :character  İst:3  1st Qu.:55.0  1st Qu.:60.0
Mode  :character      Median :72.0  Median :65.0
                        Mean   :67.4  Mean   :69.6
                        3rd Qu.:80.0  3rd Qu.:85.0
                        Max.   :90.0  Max.   :88.0

```

- Sütun ve satır seçimi, matrislerdeki gibi [satır, sütun] ya da \$ ile yapılır:

```
ogrenci$vize          # Bir sütun (vektör olarak)
```

```
[1] 55 80 40 90 72
```

```
ogrenci[, "final"]    # Aynı işlem
```

```
[1] 60 85 50 88 65
```

```
ogrenci[2, ]          # 2. satır
```

	ad <chr>	bolum <fct>	vize <dbl>	final <dbl>
2	Ayşe	Akt	80	85

1 row

```
ogrenci[ogrenci$vize > 60, ]    # Vizesi 60'tan büyük olanlar
```

	ad <chr>	bolum <fct>	vize <dbl>	final <dbl>
2	Ayşe	Akt	80	85
4	Deniz	İst	90	88
5	Ece	Akt	72	65

3 rows

```
ogrenci[ogrenci$bolum == "İst", c("ad", "vize")]
```

	ad <chr>	vize <dbl>
1	Ali	55
3	Can	40
4	Deniz	90

3 rows

```
# Yeni sütun ekleme
ogrenci$ortalama <- 0.4*ogrenci$vize + 0.6*ogrenci$final
ogrenci
```

ad	bolum	vize	final	ortalama
<chr>	<fct>	<dbl>	<dbl>	<dbl>
Ali	İst	55	60	58.0
Ayşe	Akt	80	85	83.0
Can	İst	40	50	46.0
Deniz	İst	90	88	88.8
Ece	Akt	72	65	67.8

5 rows

2.5.1 attach(), detach() ve with()

- `attach()` fonksiyonu bir data frame'in sütunlarını \$ yazmadan, doğrudan isimleriyle çağırmaı sağlar; `detach()` bu kullanımı kaldırır.

```
hesaplar <- data.frame(ev = statef, loot = incomes, shot = i or
names(hesaplar)
```

```
[1] "ev" "loot" "shot"
```

```
attach(hesaplar)
hesaplar$loot2 <- loot^2
head(hesaplar)
```

ev	loot	shot	loot2
<fct>	<dbl>	<fct>	<dbl>
1 tas	60 (55,65]		3600
2 sa	49 (45,55]		2401
3 qld	40 (35,45]		1600
4 nsw	61 (55,65]		3721
5 nsw	64 (55,65]		4096
6 nt	60 (55,65]		3600

6 rows

```
head(ev) # Çalışır
```

```
[1] tas sa qld nsw nsw nt
Levels: act nsw nt qld sa tas vic wa
```

```
head(loot) # Çalışır
```

```
[1] 60 49 40 61 64 60
```

```
# loot2 şu an çalışmaz: attach edildikten SONRA eklendi
detach(hesaplar)
attach(hesaplar) # Tekrar attach edince görünür olur
head(loot2)
```

```
[1] 3600 2401 1600 3721 4096 3600
```

```
detach(hesaplar)
```

Uyarı

`attach()` pratik görünse de aynı isimli nesneler birbirini örttüğünde karışıklığa yol açar ve `detach()` unutulunca hatalar çıkar. Bunun yerine `with()` fonksiyonu ya da 4. haftada göreceğimiz **dplyr** fonksiyonları önerilir:

```
with(hesaplar, mean(loot))
```

```
[1] 54.73333
```

```
with(ogrenci, tapply(final, bolum, mean))
```

```
Akt İst
75 66
```

2.6 Alıştırmalar (Hafta 2)

- 10'dan 100'e kadar 5'er artan bir vektör oluşturun. Kaç elemanı vardır? 50'den büyük olanları seçin.
- 3x3'lük bir birim matris oluşturun (ipucu: `diag()`).
- "düşük", "orta", "yüksek" düzeylerine sahip ordinal bir faktör oluşturup `table()` ile frekanslarını bulun.
- `ogrenci` veri çerçevesinde ortalaması 60'ın altında kalan öğrencilerin adlarını listeleyin.
- `t.test(ogrenci$vize)` sonucunu bir nesneye atayın ve `str()` ile inceleyin. Bu nesne hangi veri yapısıdır?

3 Hafta 3 — Raporlama: Quarto ve R Markdown

Bu hafta neler öğreneceğiz?

Kod, çıktı ve yorumu **tek bir belgede** birleştiren tekrarlanabilir raporlar hazırlamayı. Şu an okuduğunuz bu ders notu da bir Quarto belgesidir!

3.1 Neden Tekrarlanabilir Rapor?

Klasik yöntemde analiz R'da yapılır, sonuçlar ve grafikler kopyalanıp Word'e yapıştırılır. Veri değiştiğinde **her şeyi baştan kopyalamak** gerekir ve hata yapmak çok kolaydır.

Quarto / R Markdown ile **kod + sonuç + yorum** aynı dosyada bulunur. Veri değişirse belgeyi yeniden derlemek (*render*) yeterlidir; tüm tablolar, grafikler ve metin içindeki sayılar kendiliğinden güncellenir. Buna **tekrarlanabilir araştırma** (*reproducible research*) denir.

3.2 R Markdown ve Quarto

- **R Markdown** (`.Rmd`): 2014'ten beri kullanılan, R paketi (`rmarkdown`) tabanlı raporlama sistemi.
- **Quarto** (`.qmd`): Aynı ekibin (Posit) geliştirdiği **yeni nesil** sistem. R'ın yanında Python ve Julia'yı da destekler; ayrı bir program olarak kurulur ama **RStudio'nun güncel sürümleriyle birlikte gelir**.

İkisinin arka planda çalışma mantığı aynıdır:

`.qmd / .Rmd --(knitr: kodları çalıştırır)--> .md --(Pandoc: dönüştürür)--> HTML / PDF / Word / sunum`

Özellik	R Markdown (<code>.Rmd</code>)	Quarto (<code>.qmd</code>)
Dosya uzantısı	<code>.Rmd</code>	<code>.qmd</code>
Çıktı türü (YAML)	<code>output: html_document</code>	<code>format: html</code>
Parça seçenekleri	<code>{r, echo=FALSE}</code> başlıkta	<code># echo: false</code> parçanın içinde
Derleme	Knit düğmesi	Render düğmesi
Diller	Esas olarak R	R, Python, Julia, Observable
Uyarılar/kutular, çapraz referans	Ek paket gerekir	Yerleşik

Not

Eski `.Rmd` dosyalarınız Quarto ile de derlenebilir (`quarto render dosya.Rmd`). Bu nedenle R Markdown'ı öğrenen biri Quarto'ya çok kolay geçer. Bu derste **Quarto** kullanacağız, R Markdown yazımını da göreceğiz.

3.3 Yeni Bir Belge Oluşturma

1. **File** → **New File** → **Quarto Document...** (R Markdown için: *R Markdown...*)
2. Başlık ve yazar adını girin, çıktı türü olarak **HTML** seçin.
3. Dosyayı proje klasörüne kaydedin (ör. `hafta3.qmd`).
4. **Render** düğmesine (ya da **Ctrl/Cmd + Shift + K**) basın. Çıktı Viewer'da ya da tarayıcıda açılır.

RStudio'da **Source** (kaynak kod) ve **Visual** (Word benzeri görsel editör) olmak üzere iki düzenleme modu vardır. Visual modda tablo, resim ve başlık eklemek menülerle yapılabilir.

3.4 Bir Quarto Belgesinin Üç Parçası

Bir Quarto belgesi üç bileşenden oluşur: (1) **YAML başlığı**, (2) **Markdown metni** ve (3) **kod parçaları (chunk)**.

3.4.1 1) YAML Başlığı

Belgenin en üstünde, `---` çizgileri arasında yer alır ve belgenin **ayarlarını** içerir. YAML'da **girinti (boşluk) önemlidir**.

```
---
title: "Hafta 3 Raporu"
author: "Adınız Soyadınız"
date: today
lang: tr
format:
  html:
    toc: true          # İçindekiler tablosu
    number-sections: true
    code-fold: true    # Kodları katlanabilir yap
  docx: default        # Word çıktısı
  pdf: default         # PDF çıktısı (LaTeX gerekir, bkz. not)
execute:
  echo: true           # Kodları göster
  warning: false       # Uyarıları gösterme
---
```

Aynı başlığın **R Markdown** karşılığı:

```
---
title: "Hafta 3 Raporu"
author: "Adınız Soyadınız"
date: "2026-09-28"
output:
  html_document:
    toc: true
    number_sections: true
  word_document: default
---
```

Not

PDF çıktısı için bilgisayarda bir LaTeX dağıtımı olmalıdır. En kolay yol: R konsolunda bir kez `install.packages("tinytex"); tinytex::install_tinytex()` ya da terminalde `quarto install tinytex`.

3.4.2 2) Markdown ile Metin Biçimlendirme

Markdown, düz metne basit işaretler ekleyerek biçimlendirme yapan bir yazım dilidir.

Yazdığınız	Görünen
# Başlık 1, ## Başlık 2, ### Başlık 3	Başlık düzeyleri

kalın	kalın
<i>*italik*</i>	<i>italik</i>
`kod`	kod
* madde ya da – madde	Madde işaretli liste
1. madde	Numaralı liste
[Quarto](https://quarto.org)	<u>Quarto</u>
![Açıklama](img/r1.jpg)	Resim ekleme
$\bar{x} = \frac{1}{n} \sum x_i$	$\bar{x} = \frac{1}{n} \sum x_i$
\$\$... \$\$	Ortalanmış ayrı satırda formül

Tablo yazımı:

```
| Değişken | Tip |
|:-----|:-----|
| yas      | sayısal |
| cinsiyet | faktör  |
```

Önemli

Başlık işaretinden (#) sonra **bir boşluk** bırakılmalı, başlıkların, listelerin ve kod parçalarının önünde **bir boş satır** olmalıdır. Aksi hâlde biçimlendirme çalışmaz.

3.4.3 3) Kod Parçaları (Code Chunks)

Kod parçaları ```{r} ile başlar, ``` ile biter. RStudio'da **Ctrl + Alt + I** (Mac: **Cmd + Option + I**) kısayolu ile eklenir. Parça içindeki kod belge derlenirken çalıştırılır ve sonucu belgeye eklenir.

Quarto yazımı (seçenekler #| ile parçanın içine yazılır):

```
```{r}
#| label: özet-tablo
#| echo: false
#| fig-cap: "Arabaların hız ve fren mesafesi"
summary(cars)
plot(cars)
```
```

R Markdown yazımı (seçenekler başlığa yazılır):

```
```{r özet-tablo, echo=FALSE, fig.cap="Arabaların hız ve fren mesafesi"}
summary(cars)
plot(cars)
```
```

En sık kullanılan parça seçenekleri:

| Seçenek (Quarto) | R Markdown | Etkisi |
|------------------|------------|--------|
|------------------|------------|--------|

| | | |
|----------------|---------------|---|
| echo: false | echo=FALSE | Kodu gizle,
sonucu göster |
| eval: false | eval=FALSE | Kodu göster
ama çalıştırma |
| include: false | include=FALSE | Kodu çalıştır,
ama ne kodu
ne sonucu
göster (paket
yüklemeye için
ideal) |
| warning: false | warning=FALSE | Uyarıları gizle |
| message: false | message=FALSE | Mesajları gizle |
| fig-width: 6 | fig.width=6 | Grafik
genişliği (inç) |
| fig-cap: "..." | fig.cap="..." | Grafik başlığı |
| label: ... | {r etiket} | Parçanın adı
(Türkçe
karakter ve
boşluk
kullanmayın) |

Tüm belge için geçerli varsayılanlar YAML'daki `execute:` altında (R Markdown'da `knitr::opts_chunk$set()` ile) verilir.

3.5 Satır İçi (Inline) Kod

Metnin içine hesaplanmış bir değer yazmak için tek ters tırnak içinde `r` kullanılır:

```
Veride 50 gözlem vardır ve ortalama hız 15.4'dir.
```

Derlendiğinde şöyle görünür: Veride 50 gözlem vardır ve ortalama hız 15.4'dir. Veri değişirse bu sayılar da kendiliğinden değişir.

3.6 Quarto'ya Özgü Kullanışlı Özellikler

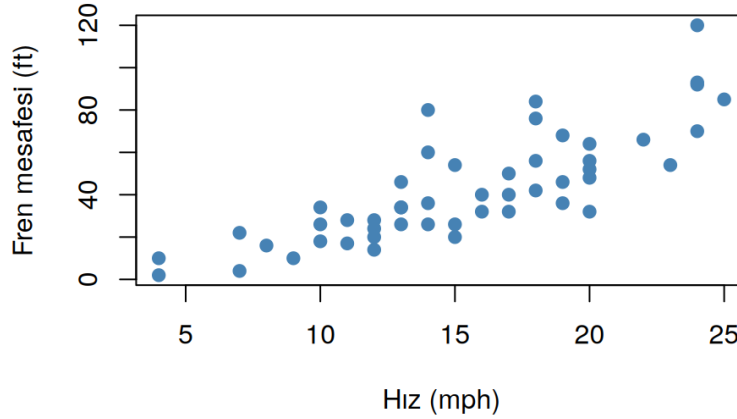
Uyarı kutuları (callout):

```
::: {.callout-tip}
## İpucu
Kodlarınızı sık sık çalıştırarak ilerleyin.
:::
```

Kutu türleri: `callout-note`, `callout-tip`, `callout-warning`, `callout-important`, `callout-caution`.

Çapraz referans: Grafiğe `fig-` ile başlayan bir etiket verip metinde `@fig-` etiket yazdığınızda numara otomatik atanır:

```
plot(cars, xlab = "Hız (mph)", ylab = "Fren mesafesi (ft)", .h
```



Şekil 8: Hız ile fren mesafesi arasındaki ilişki

Yukarıdaki kod `#| label: fig-cars` etiketine sahiptir; metinde `@fig-cars` yazıldığında Şekil 8 şeklinde görünür. Tablolar için `tbl-`, bölümler için `sec-` ön eki kullanılır.

Sekmeler (tabset):

Özet [Histogram](#)

```
summary(cars$dist)
```

| Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. |
|------|---------|--------|-------|---------|--------|
| 2.00 | 26.00 | 36.00 | 42.98 | 56.00 | 120.00 |

```
::: {.panel-tabset}
## Özet
...
## Histogram
...
:::
```

3.7 Derleme ve Sık Yapılan Hatalar

- Belge derlenirken **temiz, yeni bir R oturumu** açılır. Konsolda tanımladığınız ama belgeye yazmadığınız nesneler belgede **yoktur**. “object ‘x’ not found” hatasının en yaygın nedeni budur.
- Paketler belgenin içinde `library()` ile yüklenmelidir. `install.packages()` ise belgeye **yazılmamalıdır** (her derlemede tekrar kurmaya çalışır).
- Veri dosyaları **görelî yol** ile okunmalıdır (bkz. Çalışma dizini).
- Aynı etiket (`label`) iki parçaya verilemez.

- Etkileşimli grafikler (plotly, 6. hafta) yalnızca **HTML** çıktısında çalışır.

3.8 Alıştırmalar (Hafta 3)

1. `hafta3.qmd` adında yeni bir Quarto belgesi oluşturun; başlığa adınızı yazın, içindekiler tablosunu açın.
2. `mtcars` verisinin özetini veren bir kod parçası ekleyin; kodu gizleyip yalnızca sonucu gösterin.
3. Metin içinde satır içi kod kullanarak `mtcars` verisindeki arabaların ortalama yakıt tüketimini (`mpg`) yazdırın.
4. Belgeyi HTML ve Word olarak derleyin.

4 Hafta 4 — Veri Okuma/Yazma ve Veri Düzenleme (dplyr)

Bu hafta neler öğreneceğiz?

R'daki hazır veri setlerine ulaşmayı, dışarıdaki dosyaları (txt, csv, Excel) okuyup yazmayı ve **dplyr** paketi ile veriyi seçme, süzme, sıralama, yeni değişken türetme ve gruplara göre özetlemeyi.

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

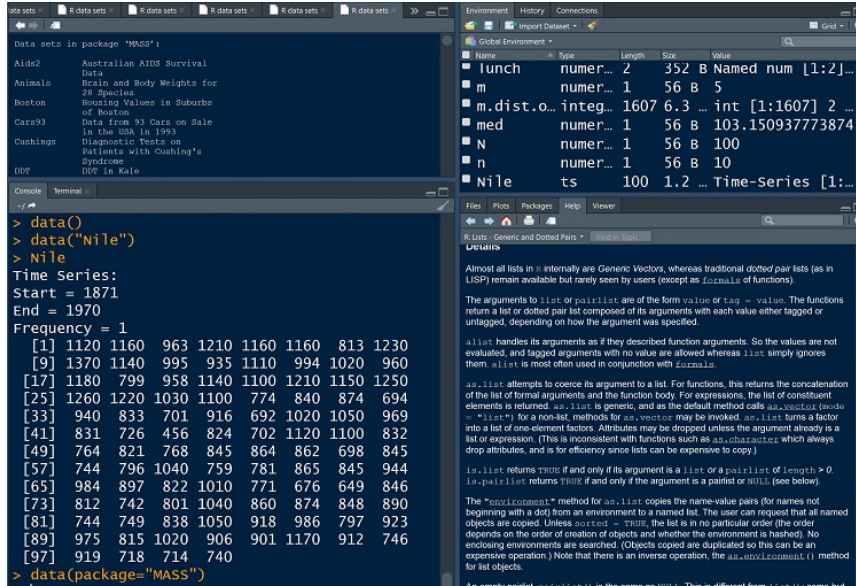
The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
library(readr)
```

4.1 Hazır Veri Setleri

- R ve kurulu paketlerin içindeki veri kümelerine `data()` fonksiyonu ile erişilebilir.



Şekil 9: data() ile hazır veri setlerini listeleme

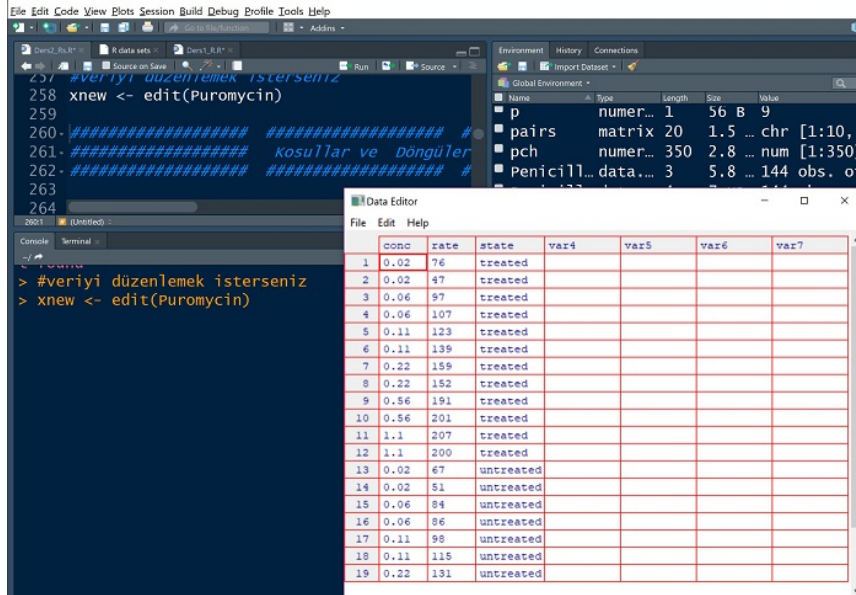
```
data() # Tüm hazır veri setlerini listeleme
data(package = "MASS") # Belli bir paketin içindeki veri setlerini listeleme
```

```
data("mtcars") # Veriyi Environment'a yükle
data(Puromycin, package = "datasets")
head(Puromycin)
```

| | conc
<dbl> | rate
<dbl> | state
<fct> |
|---|---------------|---------------|----------------|
| 1 | 0.02 | 76 | treated |
| 2 | 0.02 | 47 | treated |
| 3 | 0.06 | 97 | treated |
| 4 | 0.06 | 107 | treated |
| 5 | 0.11 | 123 | treated |
| 6 | 0.11 | 139 | treated |

6 rows

- Veriyi elle düzenlemek için `edit()` fonksiyonu kullanılabilir. Açılan tablo penceresinde değişiklik yapıp kapatıldığında düzeltilmiş veri yeni nesneye atanır.



Şekil 10: edit() ile açılan veri düzenleme penceresi

```
xnew <- edit(Puromycin) # Düzeltme girmenizi bekler (belge ç...
```

Not

edit() ile yapılan değişiklikler kodda iz bırakmaz, yani **tekrarlanabilir değildir**. Gerçek çalışmalarda düzeltmeleri kodla (ör. mutate()) yapmak tercih edilir.

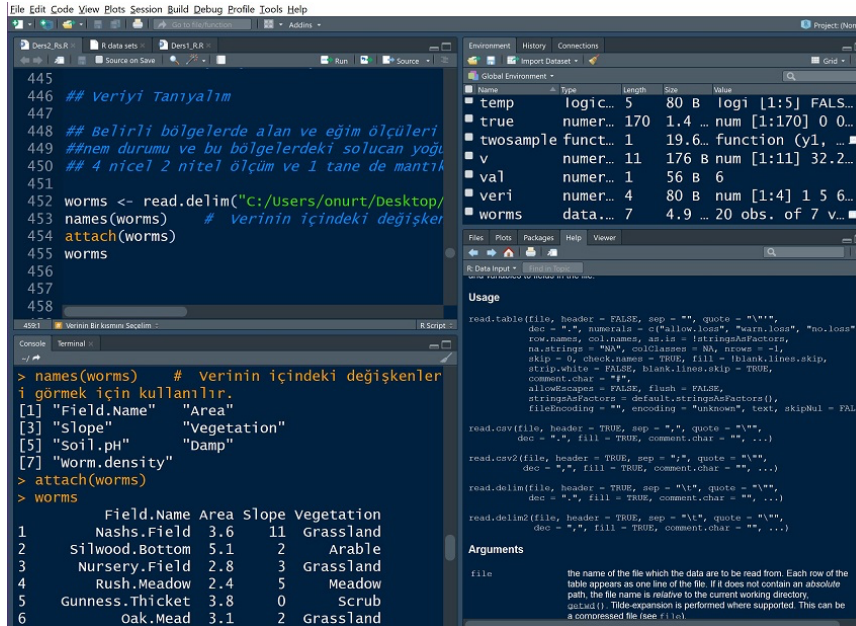
4.2 Dosyadan Veri Okuma

| Dosya türü | Base R | readr / readxl |
|--|--|----------------------|
| Sekme ile ayrılmış
(.txt, .tsv) | read.delim() /
read.table(header
= TRUE) | read_tsv() |
| Virgül ile ayrılmış
(.csv) | read.csv() | read_csv() |
| Noktalı virgül ile
ayrılmış (Türkçe
Excel'in csv'si) | read.csv2() | read_csv2() |
| Excel (.xlsx) | — | readxl::read_excel() |
| R nesnesi (.rds) | readRDS() | read_rds() |

Okumadan önce dosyayı bir metin editöründe açıp **ayırıcı karakterin** ne olduğuna (virgül, noktalı virgül, sekme), ilk satırda **değişken adları** olup olmadığına ve **ondalık ayırıcıya** (nokta mı virgül mü) bakmak gerekir.

4.2.1 Örnek 1: worms.txt

worms.txt, 20 tarladaki solucan yoğunluğunu ve tarlaların alan, eğim, bitki örtüsü, toprak pH'ı, nemlilik bilgilerini içeren küçük bir veri setidir (Crawley, *The R Book*). Değerler **sekme** ile ayrılmıştır.



Şekil 11: read.delim() ile worms verisini okuma

```
worms <- read.delim("../VERI_KUMELERI/worms.txt") # Göre
# Alternatif: read.table("../VERI_KUMELERI/worms.txt", header =
str(worms)
```

```
'data.frame': 20 obs. of 7 variables:
 $ Field.Name : chr "Nashs.Field" "Silwood.Bottom"
"Nursery.Field" "Rush.Meadow" ...
 $ Area : num 3.6 5.1 2.8 2.4 3.8 3.1 3.5 2.1 1.9 1.5
...
 $ Slope : int 11 2 3 5 0 2 3 0 0 4 ...
 $ Vegetation : chr "Grassland" "Arable" "Grassland" "Meadow"
...
 $ Soil.pH : num 4.1 5.2 4.3 4.9 4.2 3.9 4.2 4.8 5.7 5 ...
 $ Damp : logi FALSE FALSE FALSE TRUE FALSE FALSE ...
 $ Worm.density: int 4 7 2 5 6 2 3 4 9 7 ...
```

```
head(worms)
```

| Field.Name
<chr> | A...
<dbl> | Slope
<int> | Vegetation
<chr> | Soil.pH
<dbl> | Damp
<lg> |
|---------------------|---------------|----------------|---------------------|------------------|--------------|
| 1 Nashs.Field | 3.6 | 11 | Grassland | 4.1 | FALSE |
| 2 Silwood.Bottom | 5.1 | 2 | Arable | 5.2 | FALSE |
| 3 Nursery.Field | 2.8 | 3 | Grassland | 4.3 | FALSE |
| 4 Rush.Meadow | 2.4 | 5 | Meadow | 4.9 | TRUE |
| 5 Gunness.Thicket | 3.8 | 0 | Scrub | 4.2 | FALSE |
| 6 Oak.Mead | 3.1 | 2 | Grassland | 3.9 | FALSE |

6 rows

4.2.2 Örnek 2: dataset_ex.csv (müşteri kişilik analizi verisi)

Bu veri, bir şirketin 2240 müşterisinin demografik bilgilerini (doğum yılı, eğitim,

medeni durum, gelir, evdeki çocuk sayısı) ve son iki yıldaki ürün gruplarına göre harcamalarını (Mnt... sütunları) içerir.

Uzantısı .csv olmasına rağmen dosya aslında **sekme** ile ayrılmıştır. `read.csv()` ile okursak tüm satır tek bir sütuna yığılır:

```
yanlis <- read.csv("../VERI_KUMELERI/dataset_ex.csv")
dim(yanlis)      # 29 sütun olmalıydı, 1 sütun geldi!
```

```
[1] 2240      1
```

Doğru ayırıcıyı belirtince sorun çözülür:

```
musteri <- read_tsv("../VERI_KUMELERI/dataset_ex.csv", show_col = 1)
# Base R karşılığı: read.delim("../VERI_KUMELERI/dataset_ex.csv")
dim(musteri)
```

```
[1] 2240     29
```

```
glimpse(musteri)      # dplyr'ın str() benzeri özet fonksiyon.
```

```
Rows: 2,240
Columns: 29
$ ID                <dbl> 5524, 2174, 4141, 6182, 5324, 7446,
965, 6177, 485...
$ Year_Birth        <dbl> 1957, 1954, 1965, 1984, 1981, 1967,
1971, 1985, 19...
$ Education         <chr> "Graduation", "Graduation",
"Graduation", "Graduat...
$ Marital_Status    <chr> "Single", "Single", "Together",
"Together", "Marri...
$ Income            <dbl> 58138, 46344, 71613, 26646, 58293,
62513, 55635, 3...
$ Kidhome           <dbl> 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0,
0, 1, 0, 0, 1,...
$ Teenhome          <dbl> 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0,
0, 1, 0, 0, 1,...
$ Dt_Customer       <chr> "04-09-2012", "08-03-2014", "21-08-
2013", "10-02-2...
$ Recency           <dbl> 58, 38, 26, 26, 94, 16, 34, 32, 19,
68, 11, 59, 82...
$ MntWines          <dbl> 635, 11, 426, 11, 173, 520, 235,
76, 14, 28, 5, 6,...
$ MntFruits         <dbl> 88, 1, 49, 4, 43, 42, 65, 10, 0, 0,
5, 16, 61, 2, ...
$ MntMeatProducts   <dbl> 546, 6, 127, 20, 118, 98, 164, 56,
24, 6, 6, 11, 4...
$ MntFishProducts   <dbl> 172, 2, 111, 10, 46, 0, 50, 3, 3,
1, 0, 11, 225, 3...
$ MntSweetProducts  <dbl> 88, 1, 21, 3, 27, 42, 49, 1, 3, 1,
2, 1, 112, 5, 1...
$ MntGoldProds      <dbl> 88, 6, 42, 5, 15, 14, 27, 23, 2,
```

```

13, 1, 16, 30, 14...
$ NumDealsPurchases <dbl> 3, 2, 1, 2, 5, 2, 4, 2, 1, 1, 1, 1,
1, 3, 1, 1, 3,...
$ NumWebPurchases <dbl> 8, 1, 8, 2, 5, 6, 7, 4, 3, 1, 1, 2,
3, 6, 1, 7, 3,...
$ NumCatalogPurchases <dbl> 10, 1, 2, 0, 3, 4, 3, 0, 0, 0, 0,
0, 4, 1, 0, 6, 0...
$ NumStorePurchases <dbl> 4, 2, 10, 4, 6, 10, 7, 4, 2, 0, 2,
3, 8, 5, 3, 12,...
$ NumWebVisitsMonth <dbl> 7, 5, 4, 6, 5, 6, 6, 8, 9, 20, 7,
8, 2, 6, 8, 3, 8...
$ AcceptedCmp3 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
0, 0, 0, 0, 0,...
$ AcceptedCmp4 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0,...
$ AcceptedCmp5 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0,...
$ AcceptedCmp1 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 1, 0,...
$ AcceptedCmp2 <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0,...
$ Complain <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0,...
$ Z_CostContact <dbl> 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3,
3, 3, 3, 3, 3,...
$ Z_Revenue <dbl> 11, 11, 11, 11, 11, 11, 11, 11, 11, 11,
11, 11, 11, 11...
$ Response <dbl> 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
0, 0, 0, 1, 0,...

```

Not

readr fonksiyonları (read_csv , read_tsv) veriyi **tibble** olarak okur. Tibble, konsolda daha düzenli görünen modern bir data frame'dir; data frame ile yapılan her işlem tibble ile de yapılabilir.

4.3 Dosyaya Veri Yazma

```

write.csv(worms, "worms_kopya.csv", row.names = FALSE) # C
write_csv(worms, "worms_kopya.csv") # CSV
writexl::write_xlsx(worms, "worms_kopya.xlsx") # Excel
saveRDS(worms, "worms.rds") # R
worms2 <- readRDS("worms.rds")

```

4.4 dplyr ile Veri Düzenleme

dplyr, veri düzenlemeyi “fiil” gibi okunan fonksiyonlarla yapar. Her fonksiyonun ilk argümanı veridir ve sonuç yine bir veri tablosudur.

| Fonksiyon | Ne yapar? | Base R'daki karşılığı (yaklaşık) |
|-----------|-----------|----------------------------------|
|-----------|-----------|----------------------------------|

| | | |
|--------------------------|--|---|
| <code>select()</code> | Sütun seçer | <code>veri[, c("a","b")]</code> |
| <code>filter()</code> | Koşula göre satır seçer | <code>veri[veri\$a > 5,]</code> |
| <code>arrange()</code> | Satırları sıralar | <code>veri[order(veri\$a),]</code> |
| <code>mutate()</code> | Yeni değişken türetir | <code>veri\$yeni <- ...</code> |
| <code>rename()</code> | Sütun adını değiştirir | <code>names(veri)[...] <- ...</code> |
| <code>summarise()</code> | Özet değer üretir | <code>mean(), sd() ...</code> |
| <code>group_by()</code> | Sonraki işlemleri gruplara göre yapar | <code>tapply(), aggregate()</code> |
| <code>count()</code> | Frekans sayar | <code>table()</code> |

4.4.1 Pipe (Boru) Operatörü: `|>`

Birden çok işlemi art arda yapmak için **pipe** operatörü kullanılır. `x |> f(y)` ifadesi `f(x, y)` ile aynıdır ve “**sonra**” diye okunur. (RStudio kısayolu: **Ctrl/Cmd + Shift + M.**) Eski kodlarda aynı işi gören `%>%` (magrittr) operatörünü de göreceksiniz.

```
# İç içe yazım (içten dışa okunur, zor)
round(mean(worms$Worm.density), 1)
```

```
[1] 4.3
```

```
# Pipe ile (soldan sağa okunur)
worms$Worm.density |> mean() |> round(1)
```

```
[1] 4.3
```

4.4.2 `select()`: Sütun Seçme

```
worms |> select(Field.Name, Vegetation, Worm.density) |> head(3)
```

| Field.Name
<chr> | Vegetation
<chr> | Worm.density
<int> |
|---------------------|---------------------|-----------------------|
| 1 Nashs.Field | Grassland | 4 |
| 2 Silwood.Bottom | Arable | 7 |
| 3 Nursery.Field | Grassland | 2 |
| 3 rows | | |

```
worms |> select(-Field.Name) |> head(3) # Fie .M
```

| | Area
<dbl> | Slope
<int> | Vegetation
<chr> | Soil.pH
<dbl> | Damp
<lgl> | Worm.density
<int> |
|---|---------------|----------------|---------------------|------------------|---------------|-----------------------|
| 1 | 3.6 | 11 | Grassland | 4.1 | FALSE | 4 |
| 2 | 5.1 | 2 | Arable | 5.2 | FALSE | 7 |
| 3 | 2.8 | 3 | Grassland | 4.3 | FALSE | 2 |

3 rows

```
worms |> select(where(is.numeric)) |> head(3) # Say a
```

| | Area
<dbl> | Slope
<int> | Soil.pH
<dbl> | Worm.density
<int> |
|---|---------------|----------------|------------------|-----------------------|
| 1 | 3.6 | 11 | 4.1 | 4 |
| 2 | 5.1 | 2 | 5.2 | 7 |
| 3 | 2.8 | 3 | 4.3 | 2 |

3 rows

```
musteri |> select(starts_with("Mnt")) |> head(3) # "Mn
```

| MntWines
<dbl> | MntFruits
<dbl> | MntMeatProducts
<dbl> | MntFishProducts
<dbl> |
|-------------------|--------------------|--------------------------|--------------------------|
| 635 | 88 | 546 | 172 |
| 11 | 1 | 6 | 2 |
| 426 | 49 | 127 | 111 |

3 rows | 1-5 of 6 columns

4.4.3 filter(): Satır Süzme

```
worms |> filter(Vegetation == "Grassland")
```

| Field.Name
<chr> | A...
<dbl> | Slope
<int> | Vegetation
<chr> | Soil.pH
<dbl> | Damp
<lgl> |
|---------------------|---------------|----------------|---------------------|------------------|---------------|
| Nashs.Field | 3.6 | 11 | Grassland | 4.1 | FALSE |
| Nursery.Field | 2.8 | 3 | Grassland | 4.3 | FALSE |
| Oak.Mead | 3.1 | 2 | Grassland | 3.9 | FALSE |
| Church.Field | 3.5 | 3 | Grassland | 4.2 | FALSE |
| Rookery.Slope | 1.5 | 4 | Grassland | 5.0 | TRUE |
| North.Gravel | 3.3 | 1 | Grassland | 4.1 | FALSE |
| South.Gravel | 3.7 | 2 | Grassland | 4.0 | FALSE |
| Observatory.Ridge | 1.8 | 6 | Grassland | 3.8 | FALSE |
| Gravel.Pit | 2.9 | 1 | Grassland | 3.5 | FALSE |

9 rows

```
worms |> filter(Soil.pH > 5, Damp == TRUE) # Vir_ l
```

| Field.Name
<chr> | A...
<dbl> | Slope
<int> | Vegetation
<chr> | Soil.pH
<dbl> | D...
<lgl> | Worm.der
< |
|---------------------|---------------|----------------|---------------------|------------------|---------------|---------------|
|---------------------|---------------|----------------|---------------------|------------------|---------------|---------------|

```
Farm.Wood      0.8      10 Scrub      5.1 TRUE
```

1 row

```
worms |> filter(Vegetation %in% c("Arable", "Meadow")) # Bir...
```

| Field.Name
<chr> | A...
<dbl> | Slope
<int> | Vegetation
<chr> | Soil.pH
<dbl> | Damp
<lgl> | W... |
|---------------------|---------------|----------------|---------------------|------------------|---------------|------|
| Silwood.Bottom | 5.1 | 2 | Arable | 5.2 | FALSE | |
| Rush.Meadow | 2.4 | 5 | Meadow | 4.9 | TRUE | |
| Ashurst | 2.1 | 0 | Arable | 4.8 | FALSE | |
| Pond.Field | 4.1 | 0 | Meadow | 5.0 | TRUE | |
| Water.Meadow | 3.9 | 0 | Meadow | 4.9 | TRUE | |
| Pound.Hill | 4.4 | 2 | Arable | 4.5 | FALSE | |

6 rows

4.4.4 arrange(): Sıralama

```
worms |> arrange(Worm.density) |> head(3) # Küç...
```

| Field.Name
<chr> | A...
<dbl> | Slope
<int> | Vegetation
<chr> | Soil.pH
<dbl> | Damp
<lgl> | W... |
|---------------------|---------------|----------------|---------------------|------------------|---------------|------|
| 1 Observatory.Ridge | 1.8 | 6 | Grassland | 3.8 | FALSE | |
| 2 North.Gravel | 3.3 | 1 | Grassland | 4.1 | FALSE | |
| 3 Gravel.Pit | 2.9 | 1 | Grassland | 3.5 | FALSE | |

3 rows

```
worms |> arrange(desc(Worm.density)) |> head(3) # Büy...
```

| Field.Name
<chr> | A...
<dbl> | Slope
<int> | Vegetation
<chr> | Soil.pH
<dbl> | Damp
<lgl> | W... |
|---------------------|---------------|----------------|---------------------|------------------|---------------|------|
| 1 The.Orchard | 1.9 | 0 | Orchard | 5.7 | FALSE | |
| 2 Garden.Wood | 2.9 | 10 | Scrub | 5.2 | FALSE | |
| 3 Water.Meadow | 3.9 | 0 | Meadow | 4.9 | TRUE | |

3 rows

```
worms |> arrange(Vegetation, desc(Area)) |> head(5) # Önc...
```

| Field.Name
<chr> | A...
<dbl> | Slope
<int> | Vegetation
<chr> | Soil.pH
<dbl> | Damp
<lgl> | W... |
|---------------------|---------------|----------------|---------------------|------------------|---------------|------|
| 1 Silwood.Bottom | 5.1 | 2 | Arable | 5.2 | FALSE | |
| 2 Pound.Hill | 4.4 | 2 | Arable | 4.5 | FALSE | |
| 3 Ashurst | 2.1 | 0 | Arable | 4.8 | FALSE | |
| 4 South.Gravel | 3.7 | 2 | Grassland | 4.0 | FALSE | |
| 5 Nashs.Field | 3.6 | 11 | Grassland | 4.1 | FALSE | |

5 rows

4.4.5 mutate(): Yeni Değişken Türetme

```
worms |>
  mutate(yogunluk_alan = Worm.density / Area,
         asidik = ifelse(Soil.pH < 4.5, "Asidik", "Değil")) |>
  select(Field.Name, Soil.pH, asidik, yogunluk_alan) |>
  head()
```

| Field.Name
<chr> | Soil.pH
<dbl> | asidik
<chr> | yogunluk_alan
<dbl> |
|---------------------|------------------|-----------------|------------------------|
| 1 Nashs.Field | 4.1 | Asidik | 1.111111 |
| 2 Silwood.Bottom | 5.2 | Değil | 1.372549 |
| 3 Nursery.Field | 4.3 | Asidik | 0.714285 |
| 4 Rush.Meadow | 4.9 | Değil | 2.083333 |
| 5 Gunness.Thicket | 4.2 | Asidik | 1.578947 |
| 6 Oak.Mead | 3.9 | Asidik | 0.645161 |

6 rows

4.4.6 summarise() ve group_by(): Gruplara Göre Özet

```
worms |>
  summarise(n = n(),
            ort_yogunluk = mean(Worm.density),
            ss_yogunluk = sd(Worm.density))
```

| n
<int> | ort_yogunluk
<dbl> | ss_yogunluk
<dbl> |
|------------|-----------------------|----------------------|
| 20 | 4.35 | 2.621269 |

1 row

```
worms |>
  group_by(Vegetation) |>
  summarise(n = n(),
            ort_yogunluk = mean(Worm.density),
            ort_pH = mean(Soil.pH)) |>
  arrange(desc(ort_yogunluk))
```

| Vegetation
<chr> | n
<int> | ort_yogunluk
<dbl> | ort_pH
<dbl> |
|---------------------|------------|-----------------------|-----------------|
| Orchard | 1 | 9.000000 | 5.700000 |
| Meadow | 3 | 6.333333 | 4.933333 |
| Arable | 3 | 5.333333 | 4.833333 |
| Scrub | 4 | 5.250000 | 4.800000 |
| Grassland | 9 | 2.444444 | 4.100000 |

5 rows

```
worms |> count(Vegetation, Damp)
```

| Vegetation | Damp | n |
|------------|------|---|
|------------|------|---|

| <chr> | <lgl> | <int> |
|-----------|-------|-------|
| Arable | FALSE | 3 |
| Grassland | FALSE | 8 |
| Grassland | TRUE | 1 |
| Meadow | TRUE | 3 |
| Orchard | FALSE | 1 |
| Scrub | FALSE | 2 |
| Scrub | TRUE | 2 |
| 7 rows | | |

4.4.7 Uygulama: Müşteri Verisini Analize Hazırlama

Gerçek veriler çoğu zaman “kirlidir”. `musteri` verisinde bunu görelim:

```
musteri |> count(Marital_Status) # "Absurd", "YOLO", "
```

| Marital_Status | n |
|----------------|-------|
| <chr> | <int> |
| Absurd | 2 |
| Alone | 3 |
| Divorced | 232 |
| Married | 864 |
| Single | 480 |
| Together | 580 |
| Widow | 77 |
| YOLO | 2 |
| 8 rows | |

```
summary(musteri$Income) # 24 eksik gelir v. 66
```

| | | | | | | |
|------|---------|--------|-------|---------|--------|------|
| Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. | NA's |
| 1730 | 35303 | 51382 | 52247 | 68522 | 666666 | 24 |

```
range(musteri$Year_Birth) # 1893 doğumlu müş r:
```

```
[1] 1893 1996
```

Bu sorunları adım adım düzelterek analize hazır bir veri oluşturalım:

```
musteri_temiz <- muster_i |>
  filter(!is.na(Income), # Geliri eksik olanları
         Income < 200000, # Aşırı değeri çıkar
         Year_Birth >= 1940) |> # Mantıksız doğum yıl
  mutate(
    Yas = 2014 - Year_Birth, # Veri 2014 yılına ait
    Cocuk = Kidhome + Teenhome,
    Toplam_Harcama = MntWines + MntFruits + MntMeatProducts +
      MntFishProducts + MntSweetProducts + MntGroceries,
    Medeni = case_when(
```

```

    Marital_Status %in% c("Married", "Together") ~ "Birlikte",
    TRUE ~ "Yalnız"
  ),
  Egitim = factor(Education,
                  levels = c("Basic", "2n Cycle", "Graduation",
                             "Master", "PhD"),
                  ordered = TRUE)
) |>
select(ID, Yas, Egitim, Medeni, Income, Cocuk, Toplam_Harcama)

dim(musteri_temiz)

```

```
[1] 2212    13
```

```
head(musteri_temiz)
```

| ID | Y.. | Egitim | Medeni | Income | Coc... | Toplam_Harcama |
|-------|-------|------------|----------|--------|--------|----------------|
| <dbl> | <dbl> | <ord> | <chr> | <dbl> | <dbl> | <dbl> |
| 5524 | 57 | Graduation | Yalnız | 58138 | 0 | 161 |
| 2174 | 60 | Graduation | Yalnız | 46344 | 2 | 2 |
| 4141 | 49 | Graduation | Birlikte | 71613 | 0 | 77 |
| 6182 | 30 | Graduation | Birlikte | 26646 | 1 | 5 |
| 5324 | 33 | PhD | Birlikte | 58293 | 1 | 42 |
| 7446 | 47 | Master | Birlikte | 62513 | 1 | 71 |

6 rows | 1-9 of 13 columns

```

# Eğitim düzeyine göre ortalama gelir ve harcama
musteri_temiz |>
  group_by(Egitim) |>
  summarise(n = n(),
            ort_gelir = mean(Income),
            ort_harcama = mean(Toplam_Harcama))

```

| Egitim | n | ort_gelir | ort_harcama |
|------------|-------|-----------|-------------|
| <ord> | <int> | <dbl> | <dbl> |
| Basic | 54 | 20306.26 | 81.7963 |
| 2n Cycle | 198 | 47625.33 | 499.4899 |
| Graduation | 1115 | 52169.75 | 622.1883 |
| Master | 365 | 52917.53 | 609.7671 |
| PhD | 480 | 56088.26 | 674.2833 |

5 rows

Not

`case_when()` birden fazla koşula göre değer atamak için kullanılır; `ifelse()` 'in çok koşullu hâli gibi düşünülebilir. En sondaki `TRUE ~ ...` "yukarıdakilerin hiçbirisi değilse" anlamına gelir.

Temizlenmiş veriyi sonraki haftalarda kullanmak üzere kaydedebiliriz:

```
saveRDS(musteri_temiz, "../VERI_KUMELERI/musteri_temiz.rds")
```

4.5 Alıştırmalar (Hafta 4)

1. worms verisinde nemli (Damp == TRUE) tarlaların sayısını ve ortalama solucan yoğunluğunu bulun.
2. mtcars verisinde silindir sayısına (cyl) göre ortalama yakıt tüketimini (mpg) hesaplayın ve büyükten küçüğe sıralayın.
3. muster_i_temiz verisinde yaşı 40'tan küçük ve çocuğu olmayan müşterilerin ortalama şarap harcamasını (MntWines) bulun.
4. muster_i_temiz verisine gelir grubunu gösteren (Düşük < 35000 ≤ Orta < 70000 ≤ Yüksek) yeni bir değişken ekleyin ve count() ile frekanslarını bulun.

5 Hafta 5 — Kontrol Yapıları ve Fonksiyon Yazma

Bu hafta neler öğreneceğiz?

Koşula bağlı işlem yapmayı (if , else , ifelse), işlemleri tekrar etmeyi (for , while) ve kendi fonksiyonlarımızı yazmayı.

5.1 Koşul Yapıları

5.1.1 if / else

- Bir koşul **doğruysa** bir işlemi, **yanlışsa** başka bir işlemi yapar. Yapısı:

```
if (koşul) {
  koşul TRUE ise yapılacaklar
} else {
  koşul FALSE ise yapılacaklar
}
```

```
x <- -5
if (x < 0) {
  print("Negatif sayıdır")
} else {
  print("Pozitif sayıdır")
}
```

[1] "Negatif sayıdır"

- else if ile istenildiği kadar koşul art arda eklenebilir. Koşullar **yukarıdan aşağıya** kontrol edilir; ilk sağlanan koşulun işlemi yapılır.

```
x <- 0
if (x < 0) {
  print("Negatif sayıdır")
} else if (x > 0) {
  print("Pozitif sayıdır")
} else {
```

```
print("Sıfır")
}
```

```
[1] "Sıfır"
```

```
not <- 73
if (not >= 90) {
  harf <- "AA"
} else if (not >= 80) {
  harf <- "BA"
} else if (not >= 70) {
  harf <- "BB"
} else if (not >= 60) {
  harf <- "CB"
} else {
  harf <- "FF"
}
harf
```

```
[1] "BB"
```

Önemli

`if()` yalnızca **tek bir** TRUE/FALSE değeri kontrol eder. `if (c(3, -1) > 0)` gibi bir vektör verirsiniz R hata verir. Vektörün her elemanı için koşul kontrolü yapmak istiyorsanız `ifelse()` kullanın.

5.1.2 ifelse()

- Vektörel çalışır: koşul sağlanan elemanlar için ikinci argümanı, sağlanmayanlar için üçüncü argümanı döndürür. İç içe yazılabilir.

```
aaaa <- -9
abs1 <- ifelse(aaaa > 0, aaaa, -aaaa)
abs1
```

```
[1] 9
```

```
aaaa <- 0
abs1 <- ifelse(aaaa > 0, aaaa, -aaaa)
abs1
```

```
[1] 0
```

```
notlar <- c(45, 82, 67, 91, 30)
ifelse(notlar >= 60, "Geçti", "Kaldı")
```

```
[1] "Kaldı" "Geçti" "Geçti" "Geçti" "Kaldı"
```

```
ifelse(notlar >= 85, "Yüksek", ifelse(notlar >= 60, "Orta", "Üç"))
```

```
[1] "Düşük" "Orta" "Orta" "Yüksek" "Düşük"
```

5.2 Döngüler

5.2.1 for() Döngüsü

- Bir vektörün **her elemanı için** aynı işlemi tekrar eder. Yapısı: `for (eleman in vektör) { işlemler }`. Döngü değişkeninin indis olması gerekmez; doğrudan vektörün değerlerini de alabilir.

```
for (i in 1:5) {
  print(i^2)
}
```

```
[1] 1
[1] 4
[1] 9
[1] 16
[1] 25
```

```
x <- c(2, 5, 3, 9, 8, 11, 6)
count <- 0
for (val in x) {
  if (val %% 2 == 0) count <- count + 1 # Çift sayıları say
}
print(count)
```

```
[1] 3
```

- Döngü sonuçlarını saklamak için önce **boş bir vektör** oluşturup döngü içinde doldurmak iyi bir alışkanlıktır:

```
kareler <- numeric(5) # 5 elemanlı, sıfırlarla dolu vektör
for (i in 1:5) {
  kareler[i] <- i^2
}
kareler
```

```
[1] 1 4 9 16 25
```

```
(1:5)^2 # Aynı sonuç, vektörel işlemle
```

```
[1] 1 4 9 16 25
```

Not

R'da pek çok işlem **vektörel** olduğu için döngüye gerek kalmaz. Döngü yazmadan önce “bunu vektörel yapabilir miyim?” diye sormak iyi bir alışkanlıktır. Yine de döngüler, simülasyon gibi her adımın öncekine bağlı olduğu durumlarda vazgeçilmezdir.

- for döngüsü örneğini yorumlayınız.** Prostat kanseri verisinde (`islasso`

paketi) seminal vezikül invazyonu (svi : 0/1) gruplarına göre lweight ile lbph arasındaki ilişki ayrı ayrı çizdiriliyor:

```
library(islasso)
data(Prostate)
prostat <- Prostate[, -1]
names(prostat)

xss <- split(prostat$lweight, prostat$svi) # svi gruplarına göre
yss <- split(prostat$lbph, prostat$svi)
for (i in 1:length(yss)) {
  aa <- paste("svi =", i - 1)
  plot(xss[[i]], yss[[i]], xlab = aa, ylab = "lbph")
  abline(lsfilt(xss[[i]], yss[[i]])) # En küçük kareler
```

5.2.2 while() Döngüsü

- Koşul **sağlandığı sürece** tekrar eder. Kaç kez döneceği önceden bilinmediğinde kullanılır. Koşulu değiştiren bir adım unutulursa döngü **sonsuz** kadar döner (konsolda **Esc** ile durdurulur).

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

- break komutu ile döngü bitirilir; next ile o adım atlanıp bir sonrakine geçilir.
- Aşağıdaki iki while döngüsünün çıktılarını karşılaştırıp yorumlayınız:

```
i <- 1
while (i < 6) {
  print(i)
  if (i > 4) {
    break
  }
  i <- i + 1
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

```
i <- 1
while (i < 6) {
  if (i > 4) {
    break
  }
  print(i)
  i <- i + 1
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
```

```
for (i in 1:6) {
  if (i %% 2 == 0) next # Çiftleri atla
  print(i)
}
```

```
[1] 1
[1] 3
[1] 5
```

5.3 Fonksiyon Yazma

Aynı kodu birçok kez kopyalayıp yapıştırıyorsanız, onu bir **fonksiyon** haline getirme zamanı gelmiştir. Fonksiyonun üç parçası vardır: **girdiler (argümanlar)**, **gövde** (yapılacak işlemler) ve **çıkıtı** (döndürülen değer).

```
fonksiyon_adi <- function(arg1, arg2 = varsayılan_deger) {
  # işlemler
  return(sonuc) # return yazılmazsa son satırın değeri döndürülür
}
```

```
kare_al <- function(x) {
  x^2
}
kare_al(4)
```

```
[1] 16
```

```
kare_al(1:5)
```

```
[1] 1 4 9 16 25
```

```
# Birden fazla argüman ve varsayılan değer
standartlastir <- function(x, ortalama = mean(x), sapma = sd(x)) {
  (x - ortalama) / sapma
}
```

```
standartlastir(c(10, 20, 30))
```

```
[1] -1  0  1
```

```
standartlastir(c(10, 20, 30), ortalama = 0, sapma = 10)
```

```
[1] 1 2 3
```

```
# Birden fazla sonuç döndürmek için liste kullanılır
ozet <- function(x) {
  list(n = length(x), ortalama = mean(x), std_sapma = sd(x))
}
ozet(worms$Worm.density)
```

```
$n
```

```
[1] 20
```

```
$ortalama
```

```
[1] 4.35
```

```
$std_sapma
```

```
[1] 2.621269
```

- Fonksiyonun genel yapısı ve farklı çağırma şekilleri (argüman **sırasıyla** ya da **adıyla** verilebilir):

```
fun1 <- function(data, data.frame, graph, limit) {
  ## [fonksiyon gövdesi]
}

ans <- fun1(d, df, TRUE, 20)
ans <- fun1(d, df, graph = TRUE, limit = 20)
ans <- fun1(data = d, limit = 20, graph = TRUE, data.frame = df)
```

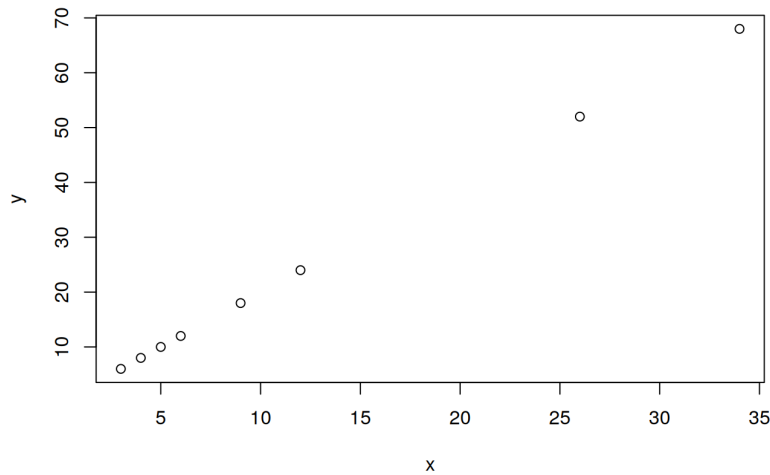
5.3.1 Örnek 1

```
x <- c(3, 5, 6, 9, 4, 12, 26, 34)

buyenifonksiyon <- function(x, grafik = TRUE) {
  y <- 2*x
  print(x)
  print(y)
  if (grafik)
    plot(x, y)
}
buyenifonksiyon(x)
```

```
[1]  3  5  6  9  4 12 26 34
```

```
[1]  6 10 12 18  8 24 52 68
```



```
buyenifonksiyon(x, grafik = FALSE)
```

```
[1] 3 5 6 9 4 12 26 34
[1] 6 10 12 18 8 24 52 68
```

5.3.2 Örnek 2: Harf Notu Fonksiyonu

Koşul yapısını fonksiyonla birleştirelim. ifelse yerine vektörel çalışan cut() kullanalım:

```
harf_notu <- function(vize, final, vize_agirlik = 0.4) {
  ort <- vize_agirlik * vize + (1 - vize_agirlik) * final
  harf <- cut(ort, breaks = c(-Inf, 60, 70, 80, 90, Inf),
             labels = c("FF", "CB", "BB", "BA", "AA"), right =
             data.frame(vize, final, ortalama = ort, harf)
}
harf_notu(vize = c(55, 80, 40, 90), final = c(60, 85, 50, 88))
```

| vize | final | ortalama | harf |
|-------|-------|----------|-------|
| <dbl> | <dbl> | <dbl> | <fct> |
| 55 | 60 | 58.0 | FF |
| 80 | 85 | 83.0 | BA |
| 40 | 50 | 46.0 | FF |
| 90 | 88 | 88.8 | BA |

4 rows

5.3.3 Örnek 3: Banka Hesabı

```
acik.hesap <- function(toplam) {
  list(
    mevduat = function(miktar) {
      if (miktar <= 0)
        stop("Mevduat pozitif olmalı!\n")
    }
  )
}
```

```

    toplam <- toplam + miktar
    cat(miktar, "yatırıldı. Bakiyeniz:", toplam, "\n\n")
  },
  cekilen = function(miktar) {      # Para çekme
    if (miktar > toplam)             # Koşul: çekilecek miktar
      stop("0 kadar para yok!\n")   # uyarı verip dur
    toplam <- toplam - miktar        # değilse miktarı düş
    cat(miktar, "çekildi. Kalan bakiyeniz:", toplam, "\n\n")
  },
  bakiye = function() {
    cat("Kalan bakiyeniz:", toplam, "\n\n")
  }
)
}

ali <- acik.hesap(100)
burak <- acik.hesap(200)
ali$cekilen(30)

```

30 çekildi. Kalan bakiyeniz: 70

```
ali$bakiye()
```

Kalan bakiyeniz: 70

```
burak$bakiye()
```

Kalan bakiyeniz: 200

```
ali$mevduat(50)
```

50 yatırıldı. Bakiyeniz: 120

```
ali$bakiye()
```

Kalan bakiyeniz: 120

```
# ali$cekilen(500)   # Hata verir. Hangi hata olduğunu bulab i
```

<- ile <<- farkı

Fonksiyon içinde <- ile yapılan atama yalnızca **fonksiyonun içinde** geçerlidir (yerel değişken). <<- ise bir üst ortamdaki değişkeni değiştirir. Yukarıda her hesabın bakiyesinin (toplam) ayrı ayrı hatırlanmasını sağlayan budur. Günlük kullanımda <<- 'dan kaçınmak önerilir.

5.4 Döngü Yerine: apply Ailesi (Kısa Bakış)

sapply() bir fonksiyonu vektörün/listenin her elemanına uygular, apply() matrisin satır (1) ya da sütunlarına (2) uygular:

```
sapply(1:5, kare_al)
```

```
[1] 1 4 9 16 25
```

```
apply(matrix(1:6, nrow = 2), 2, sum) # Sütun toplamı
```

```
[1] 3 7 11
```

```
sapply(worms[, c("Area", "Soil.pH", "Worm.density")], mean)
```

| Area | Soil.pH | Worm.density |
|-------|---------|--------------|
| 2.990 | 4.555 | 4.350 |

5.5 Alıştırmalar (Hafta 5)

1. Bir sayının tek mi çift mi olduğunu yazdıran bir fonksiyon yazın.
2. `for` döngüsü ile 1'den 100'e kadar olan sayıların toplamını bulun; sonucu `sum(1:100)` ile karşılaştırın.
3. Bir yazı turası atışında (`sample(c("Y", "T"), 1)`) art arda 3 kez yazı gelene kadar kaç atış gerektiğini `while` döngüsü ile bulun.
4. Bir sayısal vektör alıp ortalama, medyan ve değişim katsayısını (`sd/ortalama`) liste olarak döndüren bir fonksiyon yazın ve `worms` verisinin sayısal sütunlarına uygulayın.

6 Hafta 6 — Görselleştirme: base R, ggplot2 ve plotly

Bu hafta neler öğreneceğiz?

Veriyi analiz etmeden önce **görmek** gerekir. Bu hafta base R ile hızlı grafikleri, **ggplot2** ile “grafik dilbilgisi” mantığını ve **plotly** ile etkileşimli grafikleri öğreneceğiz. Amacımız süslü grafikler değil, **hangi soruya hangi grafiğin** cevap verdiğini bilmek.

```
library(ggplot2)
library(plotly)
```

Attaching package: 'plotly'

The following object is masked from 'package:ggplot2':

```
last_plot
```

The following object is masked from 'package:stats':

```
filter
```

The following object is masked from 'package:graphics':

layout

```
library(dplyr)
```

4. haftada okuduğumuz verileri kullanacağız. Bu bölüm tek başına da çalışsın diye verileri tekrar hazırlayalım:

```
worms <- read.delim("../VERI_KUMELERI/worms.txt")
musteri_temiz <- readr::read_tsv("../VERI_KUMELERI/dataset_ex.csv") %>%
  filter(!is.na(Income), Income < 200000, Year_Birth >= 1940) %>%
  mutate(Yas = 2014 - Year_Birth,
         Cocuk = Kidhome + Teenhome,
         Toplam_Harcama = MntWines + MntFruits + MntMeatProducts +
           MntFishProducts + MntSweetProducts + MntOtherProducts,
         Medeni = ifelse(Marital_Status %in% c("Married", "Together"), "Evli", "Bekar"),
         Egitim = factor(Education,
                        levels = c("Basic", "2n Cycle", "Graduate")))
```

6.1 Hangi Soruya Hangi Grafik?

| Soru | Değişken türü | Grafik |
|---|-------------------------|-----------------------------------|
| Bir sayısal değişken nasıl dağılıyor? | 1 sayısal | Histogram, yoğunluk, kutu grafiği |
| Kategorilerin frekansları nedir? | 1 kategorik | Çubuk (bar) grafiği |
| İki sayısal değişken ilişkili mi? | 2 sayısal | Serpilme (scatter) grafiği |
| Sayısal değişken gruplara göre farklı mı? | 1 sayısal + 1 kategorik | Yan yana kutu grafikleri |
| Zamanla nasıl değişiyor? | Zaman + sayısal | Çizgi grafiği |

6.2 base R Grafikleri (Hızlı Bakış)

base R grafikleri **hızlıdır** ve veriye ilk bakış için idealdir. Her grafik tek bir fonksiyonla çizilir; başlık, eksen adı ve renk argümanlarla verilir.

```
par(mfrow = c(2, 2)) # Grafik alanını 2x2'ye böl

hist(musteri_temiz$Income, main = "Histogram", xlab = "Gelir", col = "red")

boxplot(Income ~ Egitim, data = muster_i_temiz, main = "Kutu grafiği", col = "blue")
```

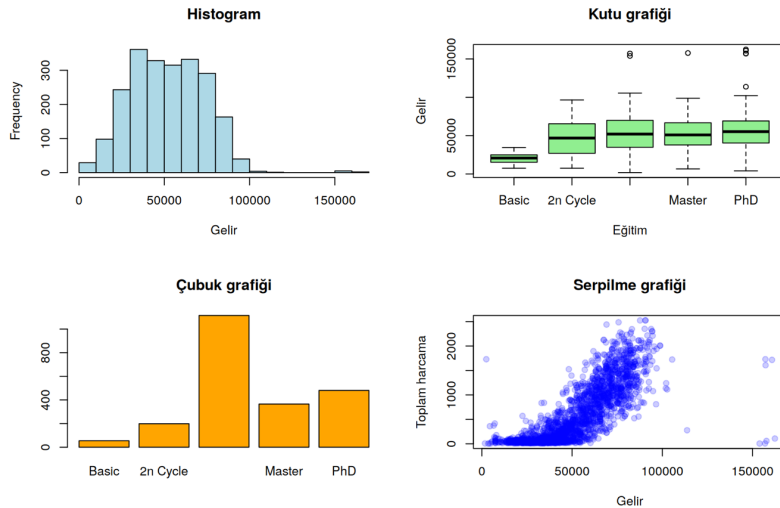
```

xlab = "Eğitim", ylab = "Gelir", col = "lightgreen")

barplot(table(musteri_temiz$Egitim), main = "Çubuk grafiği", col = "lightgreen")

plot(musteri_temiz$Income, muster_i_temiz$Toplam_Harcama, main = "Serpilme grafiği",
xlab = "Gelir", ylab = "Toplam harcama", pch = 19, col = "blue")

```



```
par(mfrow = c(1, 1)) # Eski hâline döndür
```

- Grafiği dosyaya kaydetmek için RStudio'daki **Plots** → **Export** menüsü ya da kodla `png()` ... `dev.off()` kullanılır:

```

png("gelir_histogram.png", width = 800, height = 600)
hist(musteri_temiz$Income)
dev.off()

```

6.3 ggplot2: Grafiklerin Dilbilgisi

ggplot2, grafiği katman katman kuran bir sistemdir (*Grammar of Graphics*). Her ggplot grafiği aynı üç temel bileşenden oluşur:

1. **Veri (data)**: Hangi veri tablosu?
2. **Estetik eşleme (aes())**: Hangi değişken **x**'e, hangisi **y**'ye, hangisi **renke**, **boyuta**... gidecek?
3. **Geometri (geom_...())**: Veri nokta mı, çubuk mu, kutu mu, çizgi mi olarak çizilecek?

Katmanlar + işareti ile eklenir:

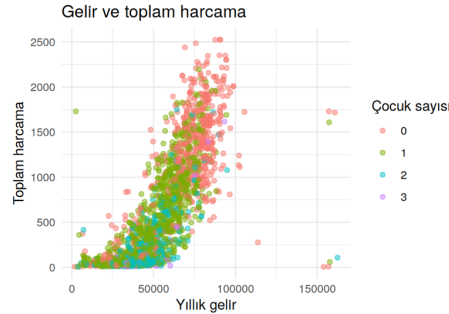
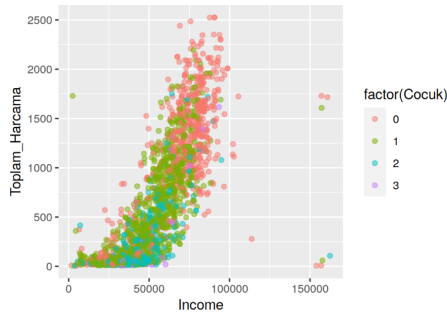
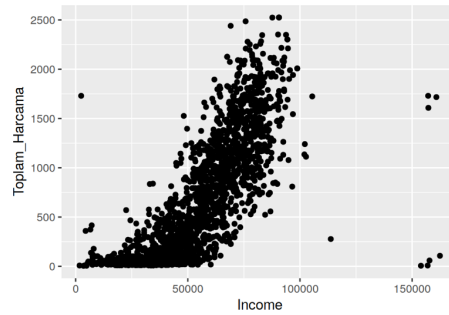
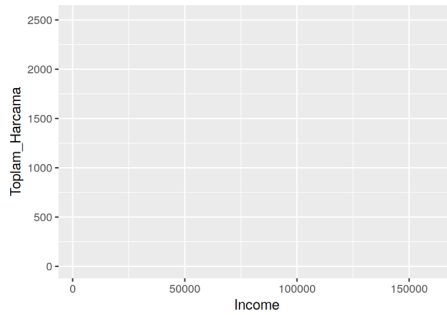
```

ggplot(data = VERİ, aes(x = DEĞİŞKEN1, y = DEĞİŞKEN2)) +
  geom_XXX() +           # geometri
  labs(...) +           # başlık ve eksen adları
  theme_XXX()           # görünüm

```

6.3.1 Adım Adım Bir ggplot

```
# 1. Sadece veri ve eşleme: boş bir tuval (eksenler var, veri yok)
ggplot(musteri_temiz, aes(x = Income, y = Toplam_Harcama))
# 2. Geometri ekleyince noktalar çizilir
ggplot(musteri_temiz, aes(x = Income, y = Toplam_Harcama)) +
  geom_point()
# 3. Rengi üçüncü bir değişkene eşleyelim ve saydamlık verelim
ggplot(musteri_temiz, aes(x = Income, y = Toplam_Harcama, colour = Toplam_Harcama)) +
  geom_point(alpha = 0.5)
# 4. Başlık, eksen adları ve tema
ggplot(musteri_temiz, aes(x = Income, y = Toplam_Harcama, colour = Toplam_Harcama)) +
  geom_point(alpha = 0.5) +
  labs(title = "Gelir ve toplam harcama",
       x = "Yıllık gelir", y = "Toplam harcama", colour = "Çocuk sayısı") +
  theme_minimal()
```



Eşleme (mapping) ile sabit değer (setting) farkı

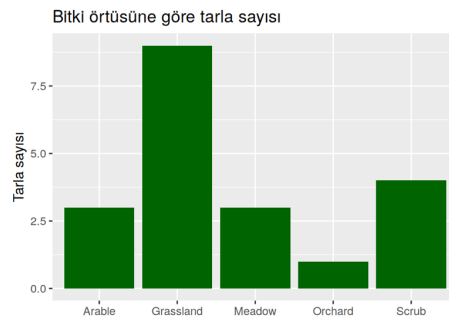
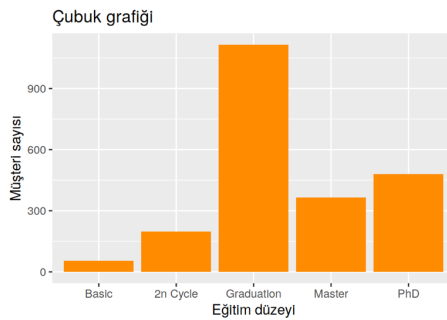
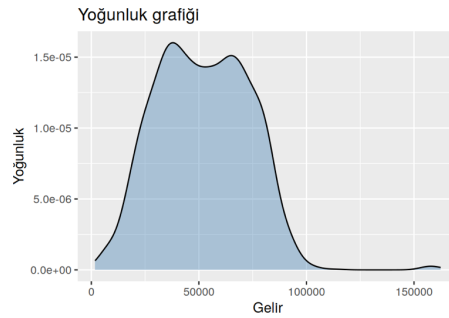
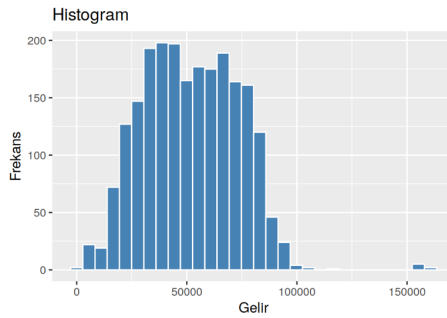
- `aes(colour = Egitim)` → Renk **veriye göre değişir** (her eğitim düzeyi farklı renk). Değişken adı `aes()` **içine** yazılır.
- `geom_point(colour = "blue")` → **Tüm noktalar** mavi olur. Sabit değer `aes()` **dışına** yazılır.

`aes(colour = "blue")` yazmak yaygın bir hatadır: R “blue” adında tek kategorili bir değişken varmış gibi davranır ve noktalar pembe çıkar!

6.3.2 Tek Değişkenin Dağılımı

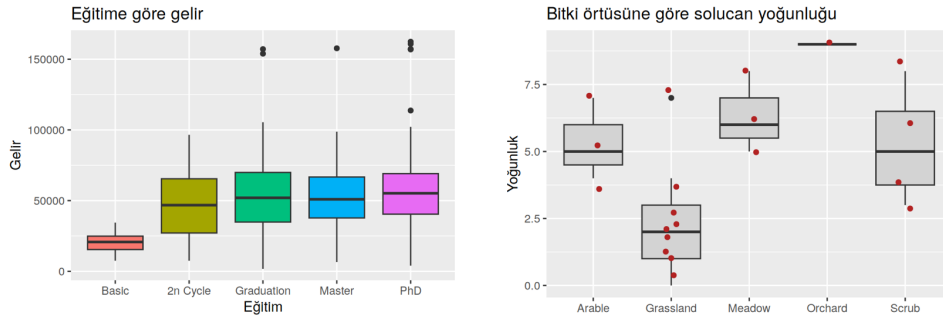
```
ggplot(musteri_temiz, aes(x = Income)) +
  geom_histogram(bins = 30, fill = "steelblue", colour = "white")
```

```
labs(title = "Histogram", x = "Gelir", y = "Frekans")
ggplot(musteri_temiz, aes(x = Income)) +
  geom_density(fill = "steelblue", alpha = 0.4) +
  labs(title = "Yoğunluk grafiği", x = "Gelir", y = "Yoğunluk")
ggplot(musteri_temiz, aes(x = Egitim)) +
  geom_bar(fill = "darkorange") +
  labs(title = "Çubuk grafiği", x = "Eğitim düzeyi", y = "Müşteri sayısı")
ggplot(worms, aes(x = Vegetation)) +
  geom_bar(fill = "darkgreen") +
  labs(title = "Bitki örtüsüne göre tarla sayısı", x = "Vegetation", y = "Tarla sayısı")
```



6.3.3 Gruplar Arası Karşılaştırma

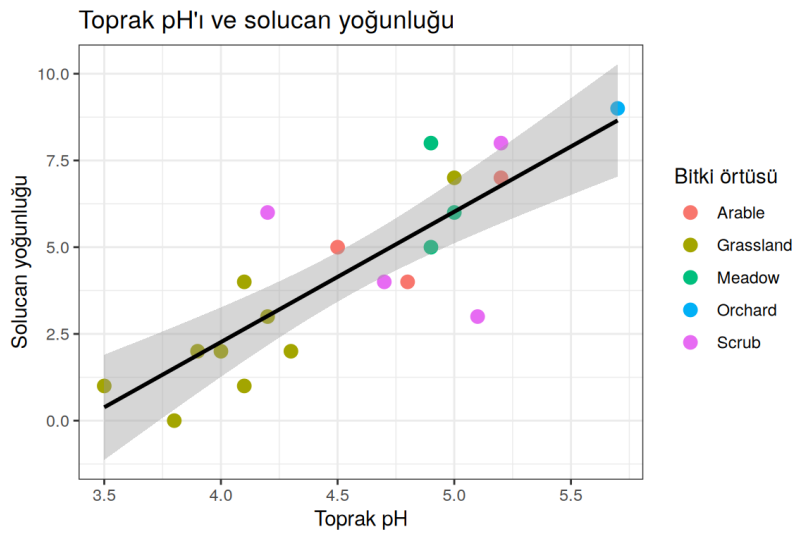
```
ggplot(musteri_temiz, aes(x = Egitim, y = Income, fill = Egitim)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Eğitime göre gelir", x = "Eğitim", y = "Gelir")
ggplot(worms, aes(x = Vegetation, y = Worm.density)) +
  geom_boxplot(fill = "lightgray") +
  geom_jitter(width = 0.1, colour = "firebrick") + # Gözlemlemler
  labs(title = "Bitki örtüsüne göre solucan yoğunluğu", x = "Vegetation", y = "Worm.density")
```



6.3.4 İki Sayısal Değişken ve Eğilim Çizgisi

```
ggplot(worms, aes(x = Soil.pH, y = Worm.density)) +
  geom_point(aes(colour = Vegetation), size = 3) +
  geom_smooth(method = "lm", se = TRUE, colour = "black") + #
  labs(title = "Toprak pH'ı ve solucan yoğunluğu",
       x = "Toprak pH", y = "Solucan yoğunluğu", colour = "Bitki",
       theme_bw())
```

`geom\_smooth()` using formula = 'y ~ x'



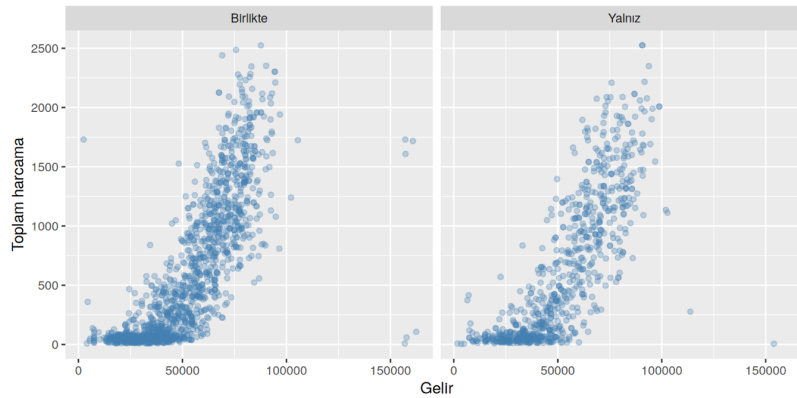
Not

`geom_smooth(method = "lm")` verideki doğrusal eğilimi gösterir. Bu çizginin nasıl hesaplandığını 13. haftada **basit doğrusal regresyon** konusunda göreceğiz.

6.3.5 Paneller (facet)

Aynı grafiği bir kategorik değişkenin her düzeyi için ayrı panelde çizmek:

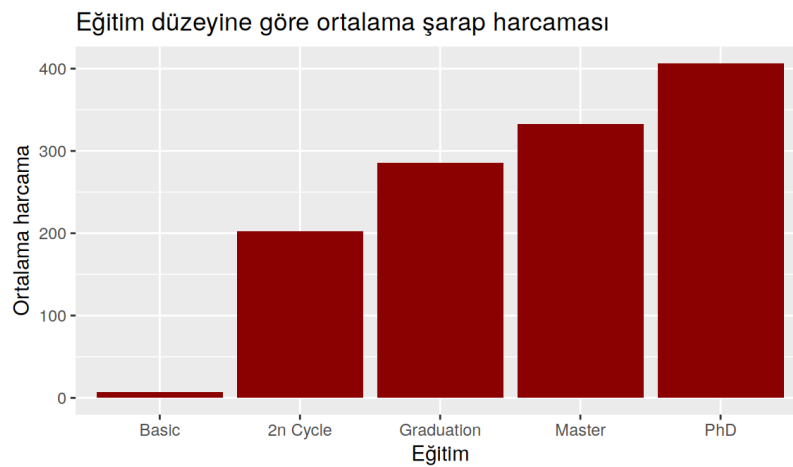
```
ggplot(musteri_temiz, aes(x = Income, y = Toplam_Harcama)) +
  geom_point(alpha = 0.3, colour = "steelblue") +
  facet_wrap(~ Medeni) +
  labs(x = "Gelir", y = "Toplam harcama")
```



6.3.6 dplyr + ggplot2 Birlikte

Özet tabloyu dplyr ile üretip doğrudan ggplot2'ye aktarabiliriz. Dikkat: dplyr adımları |> ile, ggplot katmanları + ile bağlanır.

```
musteri_temiz |>
  group_by(Egitim) |>
  summarise(ort_sarap = mean(MntWines)) |>
  ggplot(aes(x = Egitim, y = ort_sarap)) +
  geom_col(fill = "darkred") +
  labs(title = "Eğitim düzeyine göre ortalama şarap harcaması",
        x = "Eğitim", y = "Ortalama harcama")
```



6.3.7 Grafiği Kaydetme

```
g <- ggplot(worms, aes(Soil.pH, Worm.density)) + geom_point(.
ggsave("worms_ph.png", plot = g, width = 6, height = 4, dpi = 300)
```

6.4 plotly: Etkileşimli Grafikler

plotly ile çizilen grafiklerde fareyle üzerine gelindiğinde değerler görünür, yakınlaştırma yapılabilir, lejanttaki gruplar açılıp kapatılabilir. Veriyi **keşfetmek** için

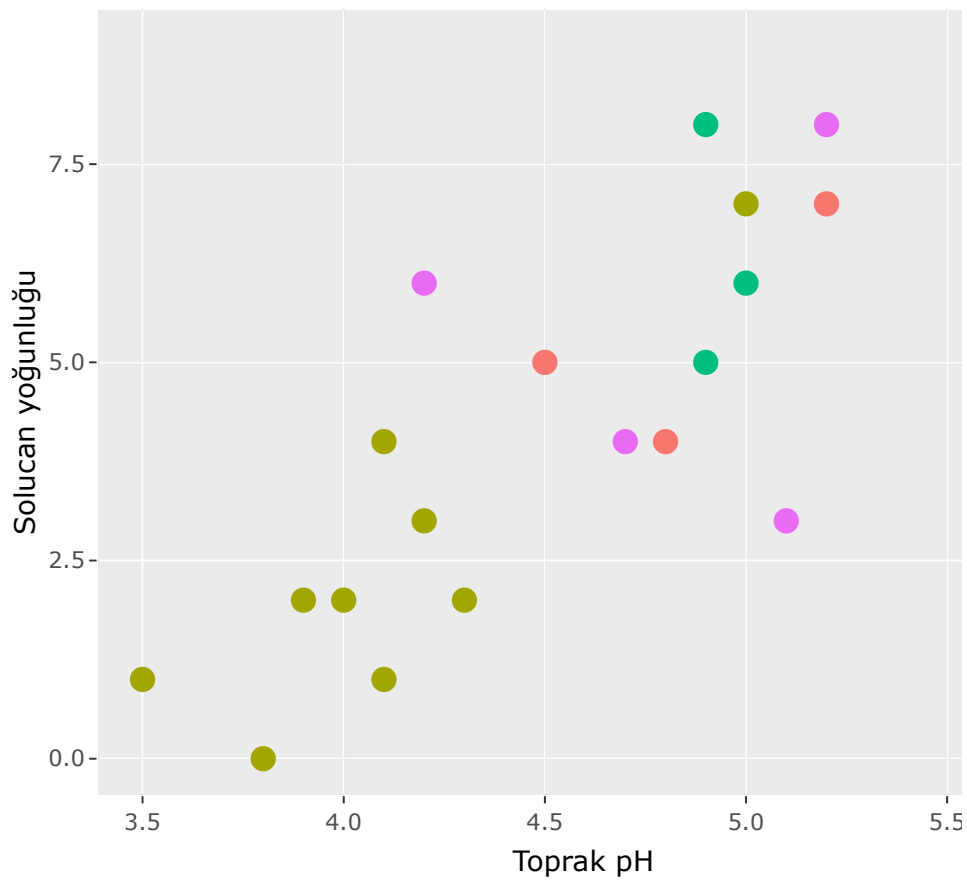
çok kullanışlıdır. Etkileşimli grafikler RStudio'nun **Viewer** penceresinde ve **HTML** çıktılarda çalışır (Word/PDF'te çalışmaz).

plotly'yi iki şekilde kullanabiliriz:

6.4.1 1) ggplotly(): Hazır ggplot'u Etkileşimli Yapmak

En kolay yol: Bildiğimiz ggplot grafiğini ggplotly() içine vermek.

```
g <- ggplot(worms, aes(x = Soil.pH, y = Worm.density, colour = Field.Name, text = Field.Name)) +
  geom_point(size = 3) +
  labs(x = "Toprak pH", y = "Solucan yoğunluğu", colour = "Bitki")
ggplotly(g) # Noktaların üzerine gelip tarla adlarını görün
```



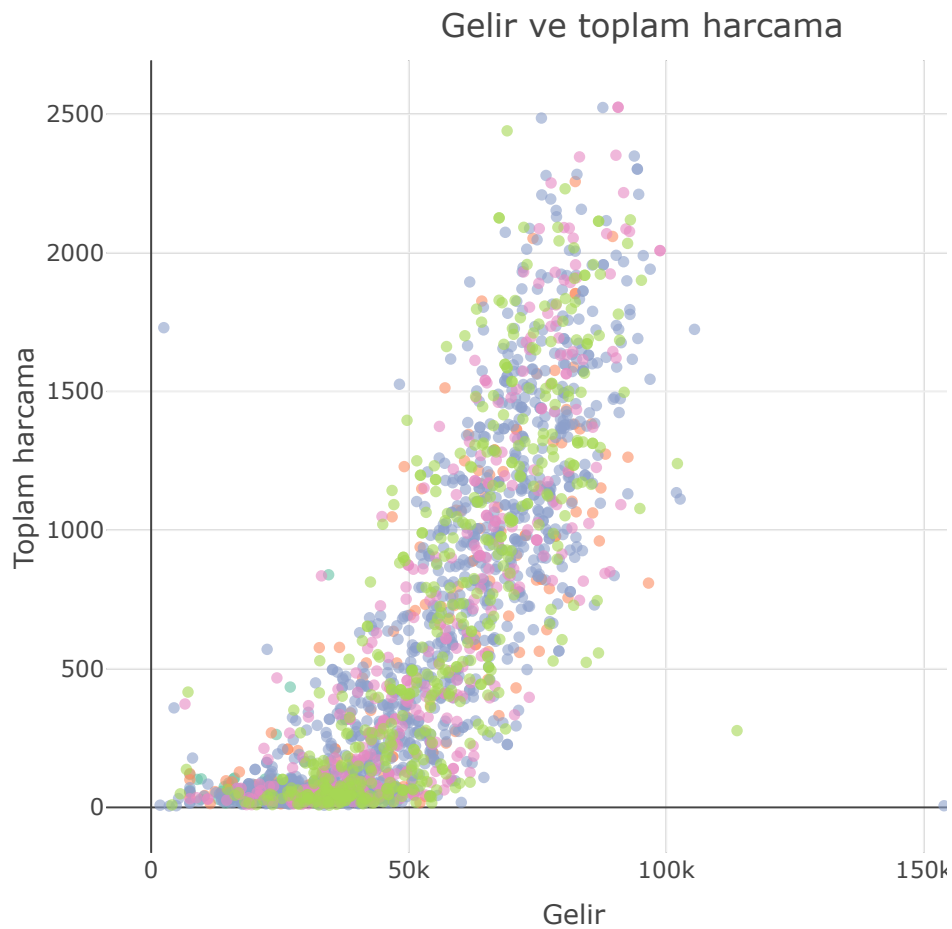
6.4.2 2) plot_ly(): Doğrudan plotly Sözdizimi

plot_ly() fonksiyonunda değişkenler başlarına **tilde (~)** konularak verilir; grafik türü type (ve gerekirse mode) argümanı ile seçilir. Katmanlar + ile değil, **pipe** (|>) ile eklenir.

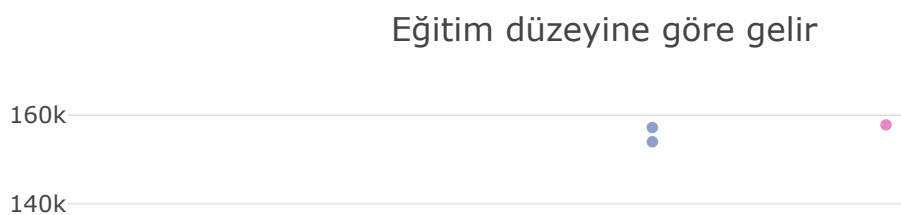
| ggplot2 | plotly |
|-----------------|------------------------------------|
| aes(x = Income) | x = ~Income |
| geom_point() | type = "scatter", mode = "markers" |

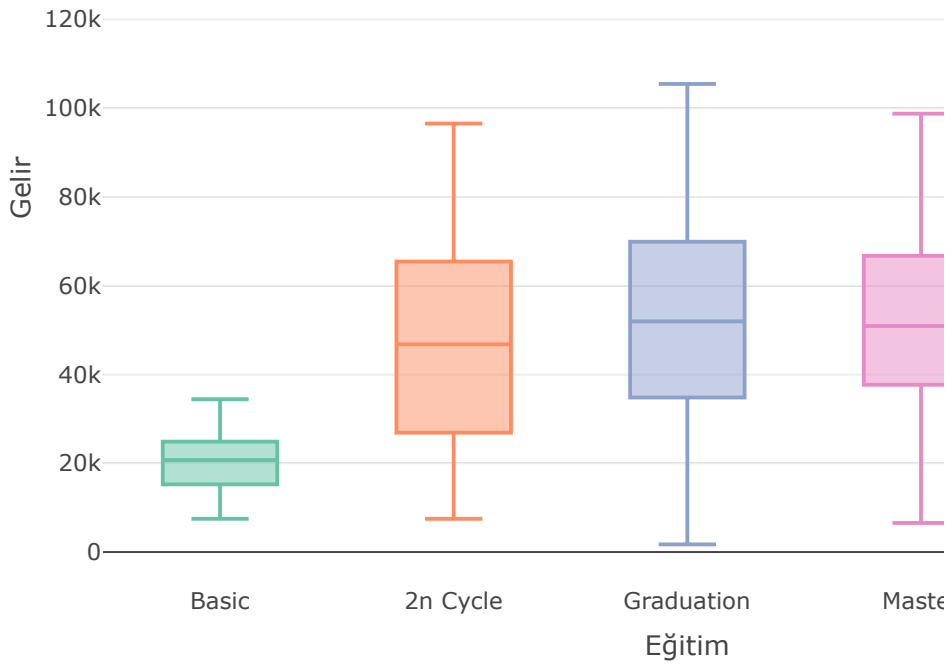
| | |
|--|---|
| <code>geom_histogram()</code> | <code>type = "histogram"</code> |
| <code>geom_boxplot()</code> | <code>type = "box"</code> |
| <code>geom_bar()</code> /
<code>geom_col()</code> | <code>type = "bar"</code> |
| <code>labs(...)</code> | <code>layout(title = ..., xaxis = list(title = ...))</code> |

```
plot_ly(musteri_temiz, x = ~Income, y = ~Toplam_Harcama, color = ~Age,
        type = "scatter", mode = "markers", alpha = 0.6,
        text = ~paste("Yaş:", Yas, "<br>Çocuk:", Cocuk)) |>
  layout(title = "Gelir ve toplam harcama",
        xaxis = list(title = "Gelir"),
        yaxis = list(title = "Toplam harcama"))
```

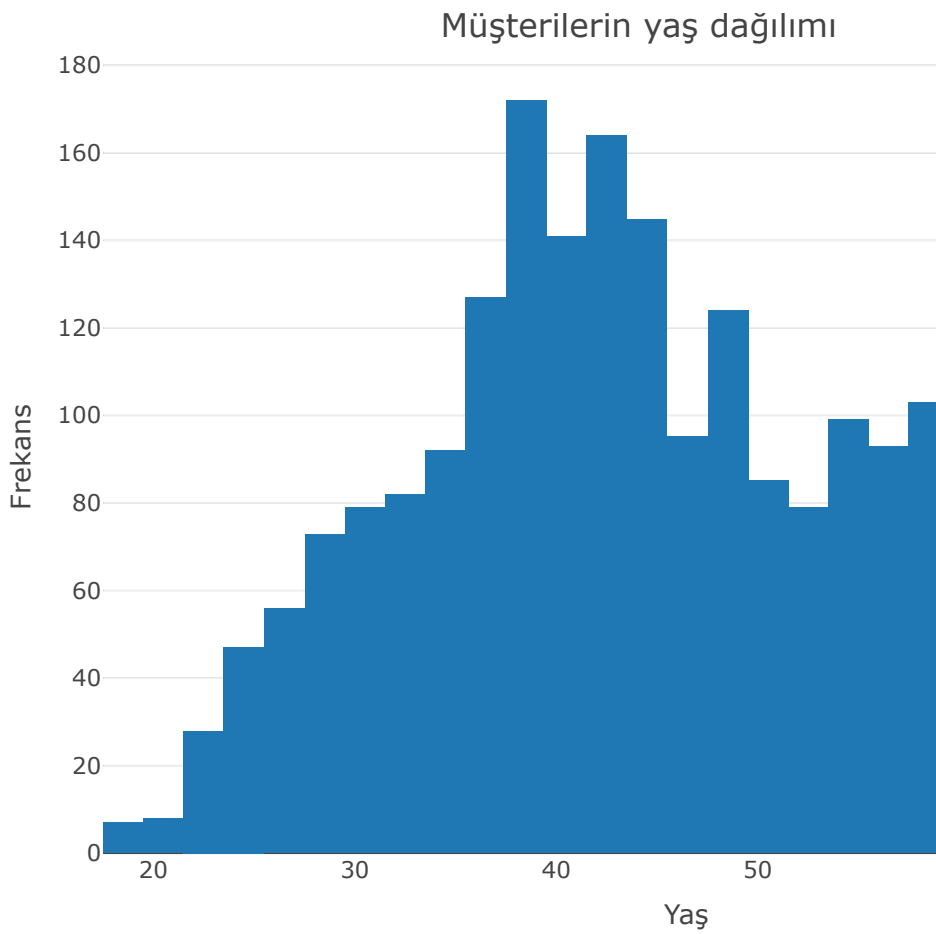


```
plot_ly(musteri_temiz, x = ~Egitim, y = ~Income, type = "box", color = ~Age,
        layout(title = "Eğitim düzeyine göre gelir", showlegend = FALSE),
        xaxis = list(title = "Eğitim"), yaxis = list(title = "Gelir"))
```





```
plot_ly(musteri_temiz, x = ~Yas, type = "histogram", nbinsx = 30,
        layout(title = "Müşterilerin yaş dağılımı",
               xaxis = list(title = "Yaş"), yaxis = list(title = "Frekans"))
```



Hangisini ne zaman kullanalım?

- **Rapor / makale / sunum** için statik grafik → **ggplot2**
- **Veriyi keşfetmek**, aykırı gözlemleri bulmak, HTML rapor ya da 14. haftada

göreceğimiz **Shiny** uygulamaları → **plotly** (çoğu zaman `ggplotly()` yeterlidir)

6.5 Alıştırmalar (Hafta 6)

1. `mtcars` verisinde ağırlık (`wt`) ile yakıt tüketimi (`mpg`) arasındaki ilişkiyi, silindir sayısına (`cyl`) göre renklendirilmiş bir `ggplot` serpilme grafiği ile gösterin. (İpucu: `factor(cyl)`)
2. Aynı grafiği `ggplotly()` ile etkileşimli hâle getirin.
3. `musteri_temiz` verisinde medeni duruma göre toplam harcamanın kutu grafiğini `plot_ly()` ile çizin.
4. `worms` verisinde alan (`Area`) ile solucan yoğunluğu arasındaki ilişkiyi nemliliğe (`Damp`) göre iki panelde (`facet_wrap`) gösterin. Ne gözlemliyorsunuz?

7 Kaynaklar

- Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2. baskı). <https://r4ds.hadley.nz>
- Crawley, M. J. (2012). *The R Book* (2. baskı). Wiley.
- Wiley, M., & Wiley, J. F. (2019). *Advanced R Statistical Programming and Data Models*. Apress.
- Quarto belgeleri: <https://quarto.org/docs/guide/>
- ggplot2 belgeleri: <https://ggplot2.tidyverse.org>
- plotly R belgeleri: <https://plotly.com/r/>